

目录

- [4. 接口说明](#)
 - [4.1 API列表](#)
 - [4.2 接口详情](#)
 - [模块概览](#)
 - [模块公共说明](#)
 - [4.2.1 运动控制模块](#)
 - [4.2.1.1 运控高层接口](#)
 - [运动模式控制](#)
 - [【服务】查询运动模式服务](#)
 - [【服务】设置运动模式服务](#)
 - [【服务】设置全身动作服务](#)
 - [预设动作控制](#)
 - [【服务】预设动作控制服务](#)
 - [输入源控制](#)
 - [【服务】设置输入源服务](#)
 - [【服务】获取当前控制输入源服务](#)
 - [4.2.1.2 运控低层接口](#)
 - [关节控制](#)
 - [【话题】关节控制话题](#)
 - [【话题】关节状态话题](#)
 - [4.2.2 交互控制模块](#)
 - [4.2.2.1 语音控制](#)
 - [音量控制服务](#)
 - [【服务】获取音量服务](#)
 - [【服务】设置音量服务](#)
 - [【服务】获取静音状态服务](#)
 - [【服务】设置静音服务](#)
 - [语音合成服务](#)
 - [【服务】语音合成服务](#)
 - [音频文件播放服务](#)
 - [【服务】音频文件播放服务](#)
 - [音频流采集话题](#)
 - [【话题】音频码流话题](#)
 - [4.2.2.2 屏幕控制](#)
 - [表情与视频管理](#)
 - [【服务】播放表情和视频服务](#)
 - [4.2.3 硬件抽象模块](#)
 - [4.2.3.1 触觉传感器](#)
 - [【话题】头部触摸状态话题](#)
 - [4.2.3.2 视觉传感器](#)

- [【服务】获取相机单帧图像服务](#)
 - [【话题】获取相机视频流服务](#)
- [4.2.3.3 电源管理](#)
 - [【话题】电源管理话题](#)
- [4.2.3.4 灯语控制](#)
 - [【服务】灯带控制服务](#)
- [4.3 下级协议定义](#)
 - [4.3.1 公共类型定义](#)
 - [4.3.2 运动控制模块](#)
 - [4.3.2.1 运控高层接口](#)
 - [4.3.2.2 运控低层接口](#)
 - [4.3.3 交互控制模块](#)
 - [4.3.3.1 语音控制](#)
 - [4.3.3.2 屏幕控制](#)
 - [4.3.4 硬件抽象模块](#)
 - [4.3.4.1 触觉传感器](#)
 - [4.3.4.2 视觉传感器](#)
 - [4.3.4.3 电源管理](#)
 - [4.3.4.4 灯语控制](#)
- [4.4.1 运动控制模块](#)
 - [4.4.1.1 运控高层接口](#)
 - [动作控制](#)
 - [速度控制](#)
 - [预设动作设置](#)
- [4.4.2 交互控制模块](#)
 - [4.4.2.1 语音控制](#)
 - [音频播放](#)
 - [音频参数控制](#)
 - [4.4.2.2 屏幕控制](#)
 - [表情播放](#)
- [4.4.3 硬件抽象模块](#)
 - [4.4.3.1 触觉传感器](#)
 - [读取触觉状态](#)
 - [4.4.3.3 电源管理](#)
 - [读取电源状态](#)
 - [4.4.3.4 灯语控制](#)
 - [灯语控制](#)

4. 接口说明

4.1 API列表

模块分类	功能分类	接口名称	功能描述
运动控制模块	运控高层接口	/aimdk 5Fmsgs/srv/GetMcAction	查询运动状态
		/aimdk 5Fmsgs/srv/SetMcAction	设置运动状态
		/aimdk 5Fmsgs/srv/SetMcMotion	设置全身动作
		/aimdk 5Fmsgs/srv/SetMcPresetMotion	设置预设动作控制
		/aima/mc/locomotion/velocity	走跑控制
		/aimdk 5Fmsgs/srv/SetMcInputSource	设置输入源
		/aimdk 5Fmsgs/srv/GetCurrentInputSource	查询当前控制输入源
交互控制模块	运控低层接口	/aima/hal/joint/command	关节控制命令
		/aima/hal/joint/state	关节状态反馈
	语音控制	/aimdk 5Fmsgs/srv/PlayTts	语音合成服务
		/aimdk 5Fmsgs/srv/GetVolume	获取音量
		/aimdk 5Fmsgs/srv/SetVolume	设置音量
		/aimdk 5Fmsgs/srv/GetMute	获取静音状态
		/aimdk 5Fmsgs/srv/SetMute	设置静音
		/aimdk 5Fmsgs/srv/PlayAudioFile	音频文件播放
		/aima/hal/audio/capture	音频码流话题

	屏幕控制	/aimdk_5Fmsgs/srv/PlayEmotion	播放表情和视频
硬件抽象模块	触觉传感器	/aima/hal/touch/state	头部触摸状态
	视觉传感器	/aima/hal/camera/bgr_5Fcamera_5Fl/capture_5Fjpeg	获取指定相机单帧JPEG图像
		rtsp_url = "rtsp://<IP>:<Port>/live" cap = cv2.VideoCapture(rtsp_url)	RTSP流媒体服务
	电源管理	/aima/hal/pmu/state	电源管理相关数据
	灯语控制	/aimdk_5Fmsgs/srv/LedStripCommand	设置整机灯语样式

4.2 接口详情

本SDK提供了一套完整的ROS2接口，用于控制启元机器人。接口遵循ROS2标准，支持C++和Python两种编程语言，为开发者提供灵活、高效的机器人应用开发环境。

模块概览

模块公共说明

返回值状态码映射 (CommonState)

以下定义了整个二次开发 SDK 的通用返回值语义，通过 `CommonResponse.status` 取得：

状态码	枚举名	业务语义描述
1	SUCCESS	指令下发/执行成功。
2	FAILURE	指令执行失败，需查阅 <code>message</code> 字段详细说明。
3	ABORTED	任务被外部中止或抢占。
4	TIMEOUT	系统响应超时。
5	INVALID	请求参数格式错误或指令当前状态非法。
100	NOT_READY	系统正在启动初始化中。
200	PENDING	任务已排队，等待中。
400	RUNNING	任务正在实时执行。

CLI 调用说明

您可以利用 ROS 2 命令行工具对上述接口进行点对点调试：

- ****服务调用 (Service Call)**:** `ros2 service call <service_name> <service_type> '<data>'`
- ****话题发布 (Topic Pub)**:** `ros2 topic pub <topic_name> <message_type> '<data>'`
- **参数获取:**
 - **字段溯源:** 载荷中的 Key 必须与对应 `.srv` 或 `.msg` 定义中的字段名**严格一致**。若定义存在嵌套，需按层级展开。
- **填充规范:** 数值直接填写；布尔值全小写 `true/false`；字符串建议“外层单引、内层双引”（如 `'text: "..."'`）。
- **缺省占位:** 对 `RequestHeader` 等嵌套对象，建议直接使用 `{}` 占位（例如 `header: {}`）。
- **示例:** `ros2 service call /aimdk_5Fmsgs/srv/SetMcAction aimdk_msgs/srv/SetMcAction '{header: {}, command: {action: {value: 201}}}'`

4.2.1 运动控制模块

本模块负责启元机器人的运动控制，涵盖了高层运动模式（如走跑、预设动作）的全局管控指令，以及底层关节电机（角度、力位）伺服的精细化控制接口。

核心功能

- **高阶业务指令调用:** 支持通过特定宏指令（Action/Motion）一键触发机器人预设动作或轨迹。
- **运动模式与信号源管理:** 支持机器人整机运动状态（如走跑、站立）的查询切换及输入源优先级配置。
- **底层关节精细伺服控制:** 支持下发关节电机标定量（位置、力矩等），支持进行高频状态监控。

4.2.1.1 运控高层接口

本模块提供高层运动指令全局管控，包含运动模式切换、输入源控制、走跑控制及预设动作触发。

运动模式控制

模式切换类接口提供了机器人运动模式查询和切换的核心功能。下表描述了当前机型支持的动作（Action）与运动（Motion）之间的映射关系：

Action 字段枚举	Motion 字符串枚举	动作名称
PASSIVE_DEFAULT	/	零力矩急停
DAMPING_DEFAULT	/	阻尼急停
BIPED_STAND_DEFAULT	/	默认站立
STORE_DEFAULT	/	折叠收储
BIPED_WALK_RUN	/	走跑
MIMIC	INTRO_POSE1	亮相动作1

MIMIC	INTRO_POSE3	亮相动作3
MIMIC	INTRO_POSE6	亮相动作6
MIMIC	INTRO_POSE7	亮相动作7
MIMIC	NIKE_POSE	耐克
MIMIC	BOXING	拳击
MIMIC	SPINNING_CRESCENT_KICK	踢月转体
MIMIC	HEAD_SPIN	头旋
MIMIC	THOMAS_FLAIR	托马斯大回环
MIMIC	FLOOR_COMBO	小地板动作
MIMIC	STANDING_SIDE_FLIP	原地侧空翻
MIMIC	ELBOW_STAND	肘立动作
MIMIC	RUNNING_FRONT_FLIP	助跑前空翻
MIMIC	BACK_HAND_SPRING_BACK_SOMERSAULT	后手翻加后空翻
MIMIC	STANDING_TO_LYING_FACE_UP	站立到躺下
MIMIC	LYING_FACE_UP_TO_STANDING	躺下到站立
MIMIC	STANDING_TO_LYING_FACE_DOWN	站立到趴下
MIMIC	LYING_FACE_DOWN_TO_STANDING	趴下到站立
MIMIC	LYING_FACE_DOWN_TO_STANDING_BY_STANDING_UP	趴下到站立-特技版

如下是当前机型的运动模式状态切换逻辑图：

```
graph TD
    DAMPING[DAMPING_DEFAULT<br/>阻尼急停] --> PASSIVE[PASSIVE_DEFAULT<br/>零力矩急停]
    PASSIVE --> STAND[BIPED_STAND_DEFAULT<br/>默认站立]
```

```

STAND <--> STORE [STORE_DEFAULT<br/>折叠收储]

STAND --> MIMIC_LC [MIMIC_LINKCRAFT<br/>模仿动作(LC)]
MIMIC_LC --> WALK [BIPED_WALK_RUN<br/>走跑]

STAND --> WALK

WALK --> MIMIC_QY [MIMIC<br/>模仿动作]

STAND --> MIMIC_QY

MIMIC_QY --> STAND
MIMIC_QY --> WALK

subgraph "MIMIC 载荷流（仅在 MIMIC 大门打开下加载）"
    MOTIONS [● 托马斯大回环（THOMAS_FLAIR）<br/>● 头旋（HEAD_SPIN）<br/>● 拳击（BOXING）<br/>● ...其他 ONNX 模型特技]
end
MIMIC_QY -. -> MOTIONS

style MIMIC_QY fill:#e1d5e7,stroke:#9673a6
style MOTIONS fill:#fff2cc,stroke:#d6b656

```

【服务】查询运动模式服务

功能描述

查询机器人当前正在执行的运动或形态动作（Action）生命周期阶段。

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Service
- 接口名称：/aimdk_5Fmsgs/srv/GetMcAction
- 数据类型：aimdk_msgs/srv/GetMcAction
- 数据流向：Robot -> Outer (状态/查询返回)
- 调用方式：Service Call (调用/请求)
- CLI 调用：ros2 service call /aimdk_5Fmsgs/srv/GetMcAction aimdk_msgs/srv/GetMcAction '{...}'
- QoS：Reliable / Volatile / Depth=1

接口定义

- 协议路径：aimdk_msgs/mc/action/srv/GetMcAction.srv

```

# Request 部分
CommonRequest request

```

```
# 响应头
ResponseHeader header

# Action信息
McActionInfo info
```

GetMcAction 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	Request 部分
header	ResponseHeader	-	响应头
info	McActionInfo	-	Action信息

接口返回值说明

- 自定义响应结构。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

【服务】设置运动模式服务

功能描述

设置合法的运动模式（如稳定站立或走跑等）。

接口元数据

- 版本：0.1 (Beta / 开发中)
- 接口类型：Service
- 接口名称：/aimdk_5Fmsgs/srv/SetMcMotion
- 数据类型：aimdk_msgs/srv/SetMcAction
- 数据流向：Outer -> Robot (控制命令/设置)
- 调用方式：Service Call (调用/请求)
- CLI 调用：`ros2 service call /aimdk_5Fmsgs/srv/SetMcMotion aimdk_msgs/srv/SetMcAction '{...}'`
- QoS：Reliable / Volatile / Depth=1

接口定义

- 协议路径：aimdk_msgs/mc/action/srv/SetMcAction.srv

```
# 请求头
RequestHeader header

# Action命令
McActionCommand command

---

CommonResponse response
```

SetMcAction 协议定义

字段	数据类型	是否必填	描述
header	RequestHeader	选填	请求头
command	McActionCommand	必填	Action命令
response	CommonResponse	-	

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

【服务】设置全身动作服务

功能描述

向机器人下发并加载执行全身动作轨迹资源 (onnx 模型或本地 physical 磁盘绝对路径资源)。

接口元数据

- 版本: 0.6.0 (Beta / 开发中)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/SetMcMotion
- 数据类型: aimdk_msgs/srv/SetMcMotion
- 调用方式: Call (服务客户端下发)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/SetMcMotion aimdk_msgs/srv/SetMcMotion '{...}'`

- QoS: Reliable / TransientLocal / Depth=10 接口定义
- 协议路径: `aimdk_msgs/mc/motion/srv/SetMcMotion.srv`

```
# 此接口是播放全身动作接口

# 请求头
RequestHeader header

# 资源类型
int32 type

# 自有模型
int32 MIMIC_QY = 1

# 灵创模型
int32 MIMIC_LC = 2

# 全身动作名称，若motion为空，加载model中resource；反之，从本地加载执行资源。
string motion

# onnx 模型绝对路径
string model

# 资源绝对路径，比如轨迹参考
string resource

# 是否打断(给后面animation接口合并预留)
bool interrupt

---

# 响应头
CommonTaskResponse response
```

SetMcMotion 协议定义

字段	数据类型	是否必填	描述
header	RequestHeader	选填	请求头
type	int32	必填	资源类型
motion	string	必填	全身动作名称，若motion为空，加载model中resource; 反之，从本地加载执行资源。

model	string	必填	onnx 模型绝对路径
resource	string	必填	资源绝对路径，比如轨迹参考
interrupt	bool	必填	是否打断(给后面animation接口合并预留)
response	CommonTaskResponse	-	响应头

SetMcMotion 枚举字典

常量名	数值	描述
MIMIC_QY	1	自有模型
MIMIC_LC	2	灵创模型

接口返回值说明

- 返回结构: [CommonTaskResponse](#)
- 核心字段: task_id ; 状态参考 [公共状态码映射](#)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

预设动作控制

【服务】预设动作控制服务

功能描述

执行系统预设的舞蹈、鞠躬等动作接口。执行前需要确保机器人已进入稳定站立模式。

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/SetMcPresetMotion
- 数据类型: aimdk_msgs/srv/SetMcPresetMotion
- 数据流向: Outer -> Robot (控制命令)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/SetMcPresetMotion aimdk_msgs/srv/SetMcPresetMotion '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: aimdk_msgs/mc/motion/srv/SetMcPresetMotion.srv

```
# 请求头
RequestHeader header

# 控制区域
McControlArea area

# 预设动作
McPresetMotion motion

# 是否打断前一个动作
bool interrupt

# 自定义动作地址
string ani_path

---

CommonTaskResponse response
```

SetMcPresetMotion 协议定义

字段	数据类型	是否必填	描述
header	RequestHeader	选填	请求头
area	McControlArea	必填	控制区域
motion	McPresetMotion	必填	预设动作
interrupt	bool	必填	是否打断前一个动作
ani_path	string	必填	自定义动作地址
response	CommonTaskResponse	-	

接口返回值说明

- 返回结构：[CommonTaskResponse](#)
- 核心字段：`task_id`；状态参考 [公共状态码映射](#)。

补充说明

-

编程示例

- Cpp示例：[点击跳转](#)
- Python示例：[点击跳转](#)

输入源控制

【服务】设置输入源服务

功能描述

动态管理多个控制信号输入源（如手柄控制、算法控制等），并执行优先级仲裁配置。

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Service
- 接口名称：/aimdk_5Fmsgs/srv/SetMcInputSource
- 数据类型：aimdk_msgs/srv/SetMcInputSource
- 数据流向：Outer -> Robot (控制命令)
- 调用方式：Service Call (调用/请求)
- CLI 调用：ros2 service call /aimdk_5Fmsgs/srv/SetMcInputSource aimdk_msgs/srv/SetMcInputSource '{...}'
- QoS：Reliable / Volatile / Depth=1

接口定义

- 协议路径：aimdk_msgs/mc/motion/srv/SetMcInputSource.srv

```
# Request 部分
CommonRequest request

McInputAction action # ADD MODIFY DELETE DISABLE ENABLE

McInputSource input_source
---

CommonTaskResponse response
```

SetMcInputSource 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	Request 部分
action	McInputAction	必填	
input_source	McInputSource	必填	
response	CommonTaskResponse	-	

接口返回值说明

- 返回结构：[CommonTaskResponse](#)
- 核心字段：task_id；状态参考 [公共状态码映射](#)。

补充说明

- 暂无

编程示例

- Cpp示例:
- Python示例:

【服务】获取当前控制输入源服务

功能描述

查询当前处于激活状态的控制输入源。

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/GetCurrentInputSource
- 数据类型: aimdk_msgs/srv/GetCurrentInputSource
- 数据流向: Robot -> Outer (状态/查询返回)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/GetCurrentInputSource aimdk_msgs/srv/GetCurrentInputSource '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: aimdk_msgs/mc/motion/srv/GetCurrentInputSource.srv

```
# Request 部分
CommonRequest request

---

CommonTaskResponse response

McInputSource input_source
```

GetCurrentInputSource 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	Request 部分
response	CommonTaskResponse	-	
input_source	McInputSource	-	

接口返回值说明

- 返回结构: [CommonTaskResponse](#)
- 核心字段: task_id ; 状态参考 [公共状态码映射](#)。

补充说明

- 暂无

编程示例

- Cpp示例：
- Python示例：

4.2.1.2 运控低层接口

本模块提供底层关节电机伺服控制，包含关节位置/力矩命令下发与自由度状态循环上报。

关节控制

【话题】 关节控制话题

功能描述

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Topic
- 接口名称：/aima/hal/joint/command
- 数据类型： `aimdk_msgs/msg/JointCommand`
- 数据流向：Outer -> Robot (控制命令)
- 调用方式：Publish (发布/写入控制)
- CLI 调用： `ros2 topic pub /aima/hal/joint/command aimdk_msgs/msg/JointCommand '{...}'`
- QoS：Best Effort / Volatile / Depth=10
- 频率：1000 Hz

接口定义

- 协议路径： `aimdk_msgs/hal/msg/JointCommand.msg`

```
string name
float64 position
float64 velocity
float64 effort
float64 stiffness
float64 damping
```

JointCommand 协议定义

字段	数据类型	是否必填	描述
name	string	必填	
position	float64	必填	
velocity	float64	必填	

effort	float64	必填	
stiffness	float64	必填	
damping	float64	必填	

接口返回值说明

补充说明

- 暂无

编程示例

- Cpp示例：
- Python示例：

【话题】 关节状态话题

功能描述

接口元数据

- 版本： 1.0 (Stable)
- 接口类型： Topic
- 接口名称： /aima/hal/joint/state
- 数据类型： aimdk_msgs/msg/JointState
- 数据流向： Robot -> Outer (状态反馈)
- 调用方式： Subscribe (订阅/读取反馈)
- CLI 调用： ros2 topic echo /aima/hal/joint/state
- QoS： Best Effort / Volatile / Depth=10
- 频率： 500 Hz

接口定义

- 协议路径： aimdk_msgs/hal/msg/JointState.msg

```
string name
float64 position
float64 velocity
float64 effort
uint8 coil_temp
uint8 motor_temp
uint8 motor_vol
```

JointState 协议定义

字段	数据类型	是否必填	描述
name	string	必填	
position	float64	必填	

velocity	float64	必填	
effort	float64	必填	
coil_temp	uint8	必填	
motor_temp	uint8	必填	
motor_vol	uint8	必填	

接口返回值说明

补充说明

- 暂无

编程示例

- Cpp示例:
- Python示例:

4.2.2 交互控制模块

本模块提供人机交互界面，包含音量配置、文本转语音（TTS）播报以及音频采集流面板接口。

核心功能

- **音量与音频管理**：支持查询和设置整机系统主音量百分比，及获取高频原始音频流数据。
- ****语音合成播报 (TTS)****：提供文本一键转音频播放接口，支持语音语义化指令交互通道。
- **屏幕联动与表情管理**：支持通过调用特定索引，一键播放对应内置表情动画（Emotion）及视频。

4.2.2.1 语音控制

本模块提供基础语音交互控制，包含系统音量调整、TTS（文本转语音）合成播放及状态流获取。

音量控制服务

【服务】获取音量服务

功能描述

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Service
- 接口名称：/aimdk_5Fmsgs/srv/GetVolume
- 数据类型：aimdk_msgs/srv/GetVolume
- 数据流向：Robot -> Outer (状态/查询返回)
- 调用方式：Service Call (调用/请求)
- CLI 调用：`ros2 service call /aimdk_5Fmsgs/srv/GetVolume aimdk_msgs/srv/GetVolume '{...}'`
- QoS：Reliable / Volatile / Depth=1

接口定义

- 协议路径：`aimdk_msgs/hal/audio/srv/GetVolume.srv`

```
# -----
# request

# 请求头
CommonRequest request

---

# -----
# response

# 响应头
CommonResponse reponse

# 当前音量 (0~100)
uint32 audio_volume
```

GetVolume 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头
reponse	CommonResponse	-	响应头
audio_volume	uint32	-	当前音量 (0~100)

接口返回值说明

- 返回结构：[CommonResponse](#)
- 状态解析：该接口遵循 [公共状态码映射](#)，成功返回 1（SUCCESS）。

补充说明

- 暂无

编程示例

- Cpp示例：[点击跳转](#)
- Python示例：[点击跳转](#)

【服务】设置音量服务

功能描述

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Service
- 接口名称：`/aimdk_5Fmsgs/srv/SetVolume`

- 数据类型: `aimdk_msgs/srv/SetVolume`
- 数据流向: Outer -> Robot (控制命令/设置)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/SetVolume aimdk_msgs/srv/SetVolume '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: `aimdk_msgs/hal/audio/srv/SetVolume.srv`

```
# -----
# request

# 请求头
CommonRequest request

# 请求音量 (0~100)
uint32 audio_volume

---

# -----
# response

# 响应头
CommonResponse reponse

# 播放音量 (0~100)
uint32 audio_volume
```

SetVolume 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头
audio_volume	uint32	必填	请求音量 (0~100)
reponse	CommonResponse	-	响应头
audio_volume	uint32	-	播放音量 (0~100)

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 `1 (SUCCESS)`。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

【服务】获取静音状态服务

功能描述

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/GetMute
- 数据类型: aimdk_msgs/srv/GetMute
- 数据流向: Robot -> Outer (状态/查询返回)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/GetMute aimdk_msgs/srv/GetMute '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: aimdk_msgs/hal/audio/srv/GetMute.srv

```
# -----
# request

# 请求头
CommonRequest request

---

# -----
# response

# 响应头
CommonResponse reponse

# 当前静音状态
bool is_mute
```

GetMute 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头

reponse	CommonResponse	-	响应头
is_mute	bool	-	当前静音状态

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

【服务】设置静音服务

功能描述

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/SetMute
- 数据类型: `aimdk_msgs/srv/SetMute`
- 数据流向: Outer -> Robot (控制命令/设置)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/SetMute aimdk_msgs/srv/SetMute '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: `aimdk_msgs/hal/audio/srv/SetMute.srv`

```
# -----
# request

# 请求头
CommonRequest request

# 请求静音状态
# true(mute)
# false(unmute)
bool is_mute

---
```

```
# -----
# response

# 响应头
CommonResponse reponse

# 当前静音状态
bool is_mute
```

SetMute 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头
is_mute	bool	必填	false(unmute)
reponse	CommonResponse	-	响应头
is_mute	bool	-	当前静音状态

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

语音合成服务

【服务】语音合成服务

功能描述

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/PlayTts
- 数据类型: `aimdk_msgs/srv/PlayTts`
- 数据流向: Outer -> Robot (控制命令/触发播报)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/PlayTts aimdk_msgs/srv/PlayTts '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径：`aimdk_msgs/interaction/srv/PlayTts.srv`

```
CommonRequest header

PlayTtsRequest tts_req

# ----- req
---
# ----- resp

CommonResponse header

PlayTtsResponse tts_resp
```

PlayTts 协议定义

字段	数据类型	是否必填	描述
header	CommonRequest	选填	
tts_req	PlayTtsRequest	必填	
header	CommonResponse	-	
tts_resp	PlayTtsResponse	-	

接口返回值说明

- 返回结构：[CommonResponse](#)
- 状态解析：该接口遵循 [公共状态码映射](#)，成功返回 `1 (SUCCESS)`。

补充说明

- 暂无

编程示例

- Cpp示例：[点击跳转](#)
- Python示例：[点击跳转](#)

音频文件播放服务

【服务】音频文件播放服务

功能描述

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Service
- 接口名称：`/aimdk_5Fmsgs/srv/PlayAudioFile`

- 数据类型: `aimdk_msgs/srv/PlayAudioFile`
- 数据流向: Outer -> Robot (控制命令/触发播放)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/PlayAudioFile aimdk_msgs/srv/PlayAudioFile '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: `aimdk_msgs/hal/audio/srv/PlayAudioFile.srv`

```
# -----
# request

# 请求头
CommonRequest request

# 文件信息
AudioFile file

---

# -----
# response

# 响应头
CommonResponse reponse
```

PlayAudioFile 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头
file	AudioFile	必填	文件信息
reponse	CommonResponse	-	响应头

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 `1 (SUCCESS)`。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)

- Python示例: [点击跳转](#)

音频流采集话题

【话题】音频码流话题

功能描述

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Topic
- 接口名称: /aima/hal/audio/capture
- 数据类型: v
- 数据流向: Robot -> Outer (数据流采集)
- 调用方式: Subscribe (订阅/读取反馈)
- CLI 调用: `ros2 topic echo /aima/hal/audio/capture`
- QoS: Reliable / Volatile / Depth=10
- 频率: 固定采样流率

接口定义

补充说明

- 暂无

编程示例

- Cpp示例:
- Python示例:

4.2.2.2 屏幕控制

本模块提供屏幕表情内容与联动管控, 支持通过特定索引触发调用播放机器人表情动画。

表情与视频管理

【服务】播放表情和视频服务

功能描述

播放指定的机器人静态表情、动态表情序列或视频文件。

接口元数据

- 接口名称: `/aimdk_5Fmsgs/srv/PlayEmotion`
- 接口类型: Service (同步)
- 接口类型: Service
- 接口名称: /aimdk_5Fmsgs/srv/PlayEmotion
- 数据类型: `aimdk_msgs/srv/PlayEmotion`
- 数据流向: Outer -> Robot (控制命令)
- 调用方式: Service Call (调用/请求)
- CLI 调用: `ros2 service call /aimdk_5Fmsgs/srv/PlayEmotion aimdk_msgs/srv/PlayEmotion '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径: `aimdk_msgs/interaction/srv/PlayEmotion.srv`

```
# -----  
-----  
# 输入表情参数单独控制灯光播放  
# -----  
-----  
  
# -----  
# 请求头  
CommonRequest header  
  
#-----  
# 表情 id  
int32 emotion_id  
  
# 表情枚举  
int32 EMOTION_UNKNOWN = 0          # 未知  
int32 EMOTION_IDLE_BLINK = 1       # 眨眼  
  
int32 EMOTION_IDLE_CALM_1 = 10     # 平静-眼睛变化  
int32 EMOTION_IDLE_CALM_2 = 11     # 平静-眼睛变化  
int32 EMOTION_IDLE_CALM_3 = 12     # 平静-眼睛变化  
  
int32 EMOTION_IDLE_GAME = 20       # 平静-游戏  
int32 EMOTION_IDLE_CUTE_1 = 30     # 平静-卖萌  
int32 EMOTION_IDLE_CUTE_2 = 31     # 平静-卖萌  
int32 EMOTION_IDLE_CUTE_3 = 32     # 平静-卖萌  
int32 EMOTION_IDLE_CUTE_4 = 33     # 平静-卖萌  
int32 EMOTION_EYE_CLOSE = 40       # 闭上眼  
int32 EMOTION_EYE_OPEN = 50        # 睁开眼  
  
int32 EMOTION_EYE_BORING_1 = 60     # 无聊  
int32 EMOTION_EYE_ABNORMAL = 70    # 异常  
int32 EMOTION_EYE_SLEEPY = 80      # 睡着  
  
int32 EMOTION_EYE_HAPPY = 90        # 喜悦(快乐)  
  
int32 EMOTION_EYE_EXTREMEHAPPY_1 = 100 # 加倍开心  
int32 EMOTION_EYE_EXTREMEHAPPY_2 = 101 # 狂喜  
int32 EMOTION_EYE_SAD = 110         # 悲伤  
int32 EMOTION_EYE_SYMPATHY = 120    # 同情  
  
int32 EMOTION_EYE_CONFUSE = 130     # 困惑(疑惑)  
  
int32 EMOTION_EYE_SHOCK = 140       # 震惊
```

```

int32 EMOTION_EYE_ACTCUTE = 150      # 撒娇
int32 EMOTION_EYE_SERIOUS = 160      # 严肃
int32 EMOTION_EYE_THINKING = 170     # 思考
int32 EMOTION_EYE_ANGRY = 180        # 愤怒
int32 EMOTION_EYE_EXTREMEANGRY = 190 # 加倍愤怒
int32 EMOTION_EYE_ADORE = 200        # 崇拜
int32 EMOTION_EYE_EXTREMEADORE = 210 # 加倍崇拜
int32 EMOTION_EYE_CHARGE = 220       # 充电

int32 EMOTION_EYE_TIRED_1 = 230      # 疲惫

int32 EMOTION_EYE_CONFIDENT_1 = 240  # 自信

int32 EMOTION_CLAM_LISTENING = 250 # 平静-聆听态 Listening

int32 EMOTION_CLAM_THINKING = 260 # 平静-思考态 Thinking

int32 EMOTION_CLAM_SPEAKING = 270 # 平静-说话态 Speaking

# -----
## 播放模式枚举
uint8 mode

# 仅播放一次
uint8 EMOTION_MODE_ONCE = 1

# 循环播放
uint8 EMOTION_MODE_LOOP = 2

# -----
# 播放优先级
uint8 priority

# ----- req
---
# -----

# ----- resp
# 响应头
CommonResponse header

# -----
# 是否控制成功
bool success

```

PlayEmotion 协议定义

字段	数据类型	是否必填	描述
header	CommonRequest	选填	请求头
emotion_id	int32	必填	表情 id
mode	uint8	必填	播放模式枚举
priority	uint8	必填	播放优先级
header	CommonResponse	-	响应头
success	bool	-	是否控制成功

PlayEmotion 枚举字典

常量名	数值	描述
EMOTION_UNKNOWN	0 # 未知	表情枚举
EMOTION_IDLE_BLINK	1 # 眨眼	
EMOTION_IDLE_CALM_1	10 # 平静-眼睛变化	
EMOTION_IDLE_CALM_2	11 # 平静-眼睛变化	
EMOTION_IDLE_CALM_3	12 # 平静-眼睛变化	
EMOTION_IDLE_GAME	20 # 平静-游戏	
EMOTION_IDLE_CUTE_1	30 # 平静-卖萌	
EMOTION_IDLE_CUTE_2	31 # 平静-卖萌	
EMOTION_IDLE_CUTE_3	32 # 平静-卖萌	
EMOTION_IDLE_CUTE_4	33 # 平静-卖萌	
EMOTION_EYE_CLOSE	40 # 闭上眼	
EMOTION_EYE_OPEN	50 # 睁开眼	
EMOTION_EYE_BORING_1	60 # 无聊	
EMOTION_EYE_ABNORMAL	70 # 异常	
EMOTION_EYE_SLEEPY	80 # 睡着	
EMOTION_EYE_HAPPY	90 # 喜悦(快乐)	
EMOTION_EYE_EXTREMEHAPPY_1	100 # 加倍开心	
EMOTION_EYE_EXTREMEHAPPY_2	101 # 狂喜	

EMOTION_EYE_SAD	110 # 悲伤	
EMOTION_EYE_SYMPATHY	120 # 同情	
EMOTION_EYE_CONFUSE	130 # 困惑(疑惑)	
EMOTION_EYE_SHOCK	140 # 震惊	
EMOTION_EYE_ACTCUTE	150 # 撒娇	
EMOTION_EYE_SERIOUS	160 # 严肃	
EMOTION_EYE_THINKING	170 # 思考	
EMOTION_EYE_ANGRY	180 # 愤怒	
EMOTION_EYE_EXTREMEANGRY	190 # 加倍愤怒	
EMOTION_EYE_ADORE	200 # 崇拜	
EMOTION_EYE_EXTREMEADORE	210 # 加倍崇拜	
EMOTION_EYE_CHARGE	220 # 充电	
EMOTION_EYE_TIRED_1	230 # 疲惫	
EMOTION_EYE_CONFIDENT_1	240 # 自信	
EMOTION_CLAM_LISTENING	250 # 平静-聆听态 Listening	
EMOTION_CLAM_THINKING	260 # 平静-思考态 Thinking	
EMOTION_CLAM_SPEAKING	270 # 平静-说话态 Speaking	
EMOTION_MODE_ONCE	1	仅播放一次
EMOTION_MODE_LOOP	2	循环播放

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

4.2.3 硬件抽象模块

本模块起底底层感知与物理能耗数据，提供触觉状态面板、电池能耗看板及 LED 灯带语义化样式控制。

核心功能

- **底层传感状态读取**：支持高频刷新和读取触觉传感器、皮肤节点的实效触摸触发信号。
- **电源状态监控看板**：提供电池主电量、温度、健康度及充放电状态的实时能耗看板查询。
- **灯带样式语义化控制**：提供 LED 灯效果流转模式管理，支持彩虹、渐变、爆闪等视觉交互渲染。

4.2.3.1 触觉传感器

本模块提供触摸传感器的状态读取接口，支持精确检测多路皮肤节点的外界触摸触发状态。

【话题】头部触摸状态话题

功能描述

接口元数据

- 版本：1.0 (Stable)
- 接口类型：Topic
- 接口名称：/aima/hal/touch/state
- 数据类型： `aimdk_msgs/msg/TouchState`
- 数据流向：Robot -> Outer (状态反馈)
- 调用方式：Subscribe (订阅/读取反馈)
- CLI 调用： `ros2 topic echo /aima/hal/touch/state`
- QoS：Reliable / Volatile / Depth=10
- 频率：触发式上报

接口定义

- 协议路径： `aimdk_msgs/hal/msg/TouchState.msg`

```
# -----  
-----  
# 机器人触摸传感器状态  
# -----  
-----  
  
# 触摸传感器状态消息  
MessageHeader header  
  
# 触摸事件类型  
uint8 event_type  
  
# 触摸事件类型常量定义  
uint8 TOUCH_EVENT_NONE           = 0  
uint8 TOUCH_EVENT_CLICK_SINGLE   = 1  
uint8 TOUCH_EVENT_CLICK_DOUBLE   = 2  
uint8 TOUCH_EVENT_CLICK_TRIPLE   = 3  
uint8 TOUCH_EVENT_CLICK_QUAD     = 4  
uint8 TOUCH_EVENT_CLICK_QUINT    = 5  
uint8 TOUCH_EVENT_CLICK_SEXT     = 6
```

```
uint8 TOUCH_EVENT_CLICK_MULTI   = 7
uint8 TOUCH_EVENT_SLIDE_LEFT    = 8
uint8 TOUCH_EVENT_SLIDE_RIGHT   = 9
```

TouchState 协议定义

字段	数据类型	是否必填	描述
header	MessageHeader	选填	触摸传感器状态消息
event_type	uint8	必填	触摸事件类型

TouchState 枚举字典

常量名	数值	描述
TOUCH_EVENT_NONE	0	触摸事件类型常量定义
TOUCH_EVENT_CLICK_SINGLE	1	
TOUCH_EVENT_CLICK_DOUBLE	2	
TOUCH_EVENT_CLICK_TRIPLE	3	
TOUCH_EVENT_CLICK_QUAD	4	
TOUCH_EVENT_CLICK_QUINT	5	
TOUCH_EVENT_CLICK_SEXT	6	
TOUCH_EVENT_CLICK_MULTI	7	
TOUCH_EVENT_SLIDE_LEFT	8	
TOUCH_EVENT_SLIDE_RIGHT	9	

接口返回值说明

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

4.2.3.2 视觉传感器

本模块提供多目视觉传感数据的流转管控接口。（当前处于待开放状态）

【服务】获取相机单帧图像服务

功能描述

接口元数据

- 版本：0.1 (Beta / 开发中)
- 接口类型：Service
- 接口名称：/aima/hal/camera/bgr_5Fcamera_5FI/capture_5Fjpeg
- 数据类型：`aimdk_msgs/srv/CaptureJpegImage`
- 数据流向：Robot -> Outer (数据流/视频图像)
- 调用方式：Service Call (调用/请求)
- CLI 调用：`ros2 service call /aimdk_5Fmsgs/srv/CaptureJpegImage aimdk_msgs/srv/CaptureJpegImage '{...}'`
- QoS: Reliable / Volatile / Depth=1

接口定义

- 协议路径：`aimdk_msgs/srv/CaptureJpegImage.srv`

```
# -----
# request

# 请求头
CommonRequest request

# 可选项，等待新帧的超时时间，单位 ms；为 0 时使用服务端默认值
uint32 timeout_ms

---

# -----
# response

# 响应头
CommonResponse response

# JPEG 图片数据
JpegImage jpeg
```

CaptureJpegImage 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	请求头
timeout_ms	uint32	必填	可选项，等待新帧的超时时间，单位 ms；为 0 时使用服务端默认值
response	CommonResponse	-	响应头

jpeg	JpegImage	-	JPEG 图片数据
------	-----------	---	-----------

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例:
- Python示例:

【话题】获取相机视频流服务

功能描述

接口元数据

- 版本: 0.1 (Beta / 开发中)
- 接口类型: RTSP
- 数据类型: `aimdk_msgs/msg/rtsp`
- 数据流向: Robot -> Outer (数据流/视频流)
- 调用方式: RTSP流媒体

接口定义

- 协议路径: `aimdk_msgs/msg/rtsp.msg`

补充说明

- 暂无

编程示例

- Cpp示例:
- Python示例:

4.2.3.3 电源管理

本模块提供整机电源状态管控面板, 支持实时监控电量百分比、健康度及内部硬件状态。

【话题】电源管理话题

功能描述

接口元数据

- 版本: 1.0 (Stable)
- 接口类型: Topic
- 接口名称: `/aima/hal/pmu/state`
- 数据类型: `aimdk_msgs/msg/PmuState`
- 数据流向: Robot -> Outer (状态反馈)
- 调用方式: Subscribe (订阅/读取反馈)

- CLI 调用: `ros2 topic echo /aima/hal/pmu/state`
- QoS: Reliable / Volatile / Depth=10
- 频率: 1 Hz

接口定义

- 协议路径: `aimdk_msgs/msg/PmuState.msg`

```
# ----- pmu ----- #
# 版本信息
string pmu_protocol_version      # 协议版本
string pmu_hardware_version      # 硬件版本
string pmu_software_version      # 软件版本

# 布尔状态
uint32 pmu_bool_status

## TODO布尔状态位定义说明:
# bit 0: rk3588PowerGood          RK3588 电源状态
# bit 1: rk3588Monitor1          RK3588 监控状态1
# bit 2: rk3588Monitor2          RK3588 监控状态2
# bit 3: orinPowerGood            ORIN 电源状态
# bit 4: orinMonitor1            ORIN 监控状态1
# bit 5: orinMonitor2            ORIN 监控状态2
# bit 6: bus48vOverCurrent        48V 总线过流
# bit 7: bus48vOverTemperature    48V 总线过温
# bit 8: rk3588PlugDetect         RK3588 插入检测
# bit 9: orinNXPlugDetect         ORIN NX 插入检测
# bits 10~31: reserved            保留位

# 电压信息
float32 soc_5v_voltage            # Soc RK3576 5V电压(单位: V)
float32 sys_5v_voltage            # 系统 5V电压(单位: V)
float32 sys_12v_voltage           # 系统 12V电压(单位: V)
float32 fore_leg_48v_voltage       # 上肢 48V电压(单位: V)
float32 hind_leg_48v_voltage       # 下肢 48V电压(单位: V)
float32 ext_12v_voltage           # 二开 12V电压(单位: V)
float32 ext_24v_voltage           # 二开 24V电压(单位: V)

# 电流信息
float32 soc_5v_current            # Soc RK3576 5V电流(单位: A)
float32 sys_5v_current            # 系统 5V电流(单位: A)
float32 sys_12v_current           # 系统 12V电流(单位: A)
float32 fore_leg_48v_current       # 上肢 48V电流(单位: A)
float32 hind_leg_48v_current       # 下肢 48V电流(单位: A)
float32 ext_12v_current           # 二开 12V电流(单位: A)
float32 ext_24v_current           # 二开 24V电流(单位: A)
```

```

# ----- bms ----- #
# 版本信息
string bms_manufacturer      # 厂家信息
string bms_serial_number     # 序列号
string bms_protocol_version  # 协议版本
string bms_hardware_version  # 硬件版本
string bms_software_version  # 软件版本

# 状态信息
uint32 bms_status_bits      # 状态位和状态码

## 状态位和状态码说明
# bit 0: chargeFlag          充电标志
# bit 1: chargeOverCurrentFlag 充电过流标志
# bit 2: dischargeFlag       放电标志
# bit 3: dischargeOverCurrentFlag 放电过流标志
# bit 4: shortCircuitFlag     电池短路标志
# bit 5: cellOverVoltageFlag  电芯电压过压标志
# bit 6: cellUnderVoltageFlag 电芯电压欠压标志
# bit 7: batteryOverVoltageFlag 电池总电压过压标志
# bit 8: batteryUnderVoltageFlag 电池总电压欠压标志
# bit 9: cellOpenCircuitFlag  电芯检测到开路标志
# bit 10: ntcOpenCircuitFlag   温度传感器检测到开路标志
# bit 11: cellDischargeOverTemperatureFlag 电芯温度超过放电温度上限标志
# bit 12: cellChargeOverTemperatureFlag 电芯温度超过充电温度上限标志
# bit 13: cellDischargeUnderTemperatureFlag 电芯温度低于放电温度下限标志
# bit 14: cellChargeUnderTemperatureFlag 电芯温度低于充电温度下限标志
# bit 15: reserved_1         保留位
# bit 16: cellMaxVoltageDiffOverHighFlag 电芯最大电压差超过上限标志
# bit 17: mosfetChargeDisableFlag MOSFET禁止充电标志
# bit 18: mosfetDischargeDisableFlag MOSFET禁止放电标志
# bit 19: mosfetOverTemperatureFlag MOSFET温度超过工作温度上限标志
# bit 20: balanceLineResistanceOverHighFlag 电池包均衡线电阻超过上限标志
# bit 21~31: reserved_11     保留位

uint16 battery_balance_line_resistance # 电池包均衡线电阻（单位：mΩ）
float64 battery_pack_voltage           # 电池包电压（单位：V）
float64 battery_current                 # 电池包电流（单位：A，充电为正，放电
为负）
float64 battery_output_power            # 电池包输出功率（单位：W）
float64 battery_temperature             # 电池包当前温度（单位：℃）
uint32 battery_remaining_capacity       # 当前电池剩余容量（单位：mAh）
uint8 battery_remaining_capacity_percentage # 当前电池剩余容量百分比（单位：%）
uint16 battery_cycle_count              # 当前电池包完整循环次数（单位：次）
uint32 battery_cycle_total_capacity     # 总充放电容量累计（单位：Ah）

```

PmuState 协议定义

字段	数据类型	是否必填	描述
pmu_protocol_version	string	必填	版本信息
pmu_hardware_version	string	必填	
pmu_software_version	string	必填	
pmu_bool_status	uint32	必填	布尔状态
soc_5v_voltage	float32	必填	电压信息
sys_5v_voltage	float32	必填	
sys_12v_voltage	float32	必填	
fore_leg_48v_voltage	float32	必填	
hind_leg_48v_voltage	float32	必填	
ext_12v_voltage	float32	必填	
ext_24v_voltage	float32	必填	
soc_5v_current	float32	必填	电流信息
sys_5v_current	float32	必填	
sys_12v_current	float32	必填	
fore_leg_48v_current	float32	必填	
hind_leg_48v_current	float32	必填	
ext_12v_current	float32	必填	
ext_24v_current	float32	必填	
bms_manufacturer	string	必填	版本信息
bms_serial_number	string	必填	
bms_protocol_version	string	必填	
bms_hardware_version	string	必填	
bms_software_version	string	必填	
bms_status_bits	uint32	必填	状态信息

battery_balance_line_resistance	uint16	必填	bit 21~31: reserved_11 保留位
battery_pack_voltage	float64	必填	
battery_current	float64	必填	
battery_output_power	float64	必填	
battery_temperature	float64	必填	
battery_remaining_capacity	uint32	必填	
battery_remaining_capacity_percentage	uint8	必填	
battery_cycle_count	uint16	必填	
battery_cycle_total_capacity	uint32	必填	

接口返回值说明

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

4.2.3.4 灯语控制

本模块提供灯带样式与语义化流光效果控制接口，支持高阶灯光颜色与频率渲染管控。

【服务】灯带控制服务

功能描述

接口元数据

- 版本：0.1 (Beta / 开发中)
- 接口类型：Service
- 接口名称：/aimdk_5Fmsgs/srv/SetRgbStrip
- 数据类型：aimdk_msgs/srv/SetRgbStrip
- 数据流向：Outer -> Robot (控制命令)
- 调用方式：Service Call (调用/请求)
- CLI 调用：

```
ros2 service call /aimdk_5Fmsgs/srv/SetRgbStrip aimdk_msgs/srv/SetRgbStrip '{...}'
```
- QoS：Reliable / Volatile / Depth=1

接口定义

- 协议路径：aimdk_msgs/ha1/srv/SetRgbStrip.srv

```
# 灯带控制服务
CommonRequest request

# 灯带模式
uint8 led_strip_mode

## led_strip_mode: 灯带模式枚举
uint8 LED_GREEN_FLOW = 1
uint8 LED_RAINBOW = 2
uint8 LED_DANCING = 3
uint8 LED_TWINKLE = 4
uint8 LED_TAIL_INCREASE = 5
uint8 LED_TAIL_DECREASE = 6
uint8 LED_FOUR_TO_TWO_LEGS = 7
uint8 LED_TWO_TO_FOUR_LEGS = 8
uint8 LED_START_UP = 9
uint8 LED_DISPLAY_CHARGING = 10
uint8 LED_DISPLAY_FAULT = 11
uint8 LED_DISPLAY_25_LOW_BATTERY = 12
uint8 LED_DISPLAY_15_LOW_BATTERY = 13
uint8 LED_DISPLAY_POWER_OFF = 14
uint8 LED_DISPLAY_POWER_ON = 15
uint8 LED_WHITE_FLOW = 16
uint8 LED_WHITE_ON = 17
---

CommonResponse response
uint16 result    #设置是否成功
```

SetRgbStrip 协议定义

字段	数据类型	是否必填	描述
request	CommonRequest	选填	灯带控制服务
led_strip_mode	uint8	必填	灯带模式
response	CommonResponse	-	
result	uint16	-	

SetRgbStrip 枚举字典

常量名	数值	描述
LED_GREEN_FLOW	1	led_strip_mode: 灯带模式枚举

LED_RAINBOW	2	
LED_DANCING	3	
LED_TWINKLE	4	
LED_TAIL_INCREASE	5	
LED_TAIL_DECREASE	6	
LED_FOUR_TO_TWO_LEGS	7	
LED_TWO_TO_FOUR_LEGS	8	
LED_START_UP	9	
LED_DISPLAY_CHARGING	10	
LED_DISPLAY_FAULT	11	
LED_DISPLAY_25_LOW_BATTERY	12	
LED_DISPLAY_15_LOW_BATTERY	13	
LED_DISPLAY_POWER_OFF	14	
LED_DISPLAY_POWER_ON	15	
LED_WHITE_FLOW	16	
LED_WHITE_ON	17	

接口返回值说明

- 返回结构: [CommonResponse](#)
- 状态解析: 该接口遵循 [公共状态码映射](#), 成功返回 1 (SUCCESS)。

补充说明

- 暂无

编程示例

- Cpp示例: [点击跳转](#)
- Python示例: [点击跳转](#)

4.3 下级协议定义

4.3.1 公共类型定义

CommonRequest

- 协议路径: aimdk_msgs/common/CommonRequest.msg

字段	数据类型	描述
----	------	----

header	RequestHeader	
--------	-------------------------------	--

CommonResponse

- 协议路径: aimdk_msgs/common/CommonResponse.msg

字段	数据类型	描述
header	ResponseHeader	
status	CommonState	
message	string	

MessageHeader

- 协议路径: aimdk_msgs/common/MessageHeader.msg

字段	数据类型	描述
stamp	builtin_interfaces/Time	
frame_id	string	
sequence	uint32	
meas_stamp	builtin_interfaces/Time	

CommonTaskResponse

- 协议路径: aimdk_msgs/common/CommonTaskResponse.msg

字段	数据类型	描述
header	ResponseHeader	
task_id	uint64	
state	CommonState	

RequestHeader

- 协议路径: aimdk_msgs/common/RequestHeader.msg

字段	数据类型	描述
stamp	builtin_interfaces/Time	

ResponseHeader

- 协议路径: aimdk_msgs/common/ResponseHeader.msg

字段	数据类型	描述
stamp	builtin_interfaces/Time	

code	int64	
------	-------	--

CommonState

1 | SUCCESS | 指令下发/执行成功。 2 | FAILURE | 指令执行失败，需查阅 message 实意。 3 | ABORTED | 任务被外部中止或抢占失效。 4 | TIMEOUT | 系统响应超时（网络或底层控制器未及时响应）。 5 | INVALID | 请求参数格式错误或指令当前状态非法。 100 | NOT_READY | 系统正在启动初始化中。 200 | PENDING | 任务已排队，等待前置任务完成。 400 | RUNNING | 任务正在实时执行中。

- 协议路径: aimdk_msgs/common/CommonState.msg

字段	数据类型	描述
value	int32	

CommonState 枚举字典

常量名	数值	描述
UNKNOWN	0	
SUCCESS	1	
FAILURE	2	
ABORTED	3	
TIMEOUT	4	
INVALID	5	
IN_MANUAL	6	
NOT_READY	100	
PENDING	200	
CREATED	300	
RUNNING	400	

4.3.2 运动控制模块

4.3.2.1 运控高层接口

McInputSource

- 协议路径: aimdk_msgs/mc/motion/McInputSource.msg

字段	数据类型	描述
name	string	输入源名称（rc/vr/app_proxy/interaction/pnc）

priority	int32	输入源优先级(0-100)
timeout	int32	超时时间(ms), 默认 1000 ms

McActionInfo

- 协议路径: aimdk_msgs/mc/action/McActionInfo.msg

字段	数据类型	描述
current_action	McAction	
action_desc	string	
status	McActionStatus	

McControlArea

- 协议路径: aimdk_msgs/mc/motion/McControlArea.msg

字段	数据类型	描述
value	int32	

常量枚举字典

常量名	数值	描述
NONE	0	没有控制区域
LEFT_HAND	1	左手
RIGHT_HAND	2	右手
HEAD	4	头部
WAIST	8	腰部

McInputAction

- 协议路径: aimdk_msgs/mc/motion/McInputAction.msg

字段	数据类型	描述
value	int32	

常量枚举字典

常量名	数值	描述
INPUTACTION_ADD	1001	添加新的输入源
INPUTACTION_MODIFY	1002	修改输入源配置

INPUTACTION_DELETE	1003	删除输入源
INPUTACTION_ENABLE	2001	使能输入源
INPUTACTION_DISABLE	2002	下使能输入源

McPresetMotion

- 协议路径: aimdk_msgs/mc/motion/McPresetMotion.msg

字段	数据类型	描述
value	int32	

常量枚举字典

常量名	数值	描述
RIGHT_ARM_WAVE	1001	右手挥手
RIGHT_ARM_HANDSHAKE	1002	右手握手
RIGHT_ARM_HANDSHAKE_FOR_1D5METER	1003	左手握手1.5m上对象
LEFT_ARM_WAVE	1011	左手挥手
LEFT_ARM_HANDSHAKE	1012	左手握手
LEFT_ARM_HANDSHAKE_FOR_1D5METER	1013	左手握手1.5m上对象
BOTH_HANDS_MAKE_HEART	2001	双手比心
BOTH_HANDS_ACTION_HAPPY	2002	双手摆动, 表达喜悦
BOTH_HANDS_ACTION_TALKING	2003	双手动作, 说话动作
PHOTO_POSE_1	2011	拍照动作1
PHOTO_POSE_2	2012	拍照动作2
PHOTO_POSE_3	2013	拍照动作3
PHOTO_POSE_4	2014	拍照动作4
PHOTO_THREE_SHOT	2020	三连拍姿势

McActionCommand

- 协议路径: aimdk_msgs/mc/action/McActionCommand.msg

字段	数据类型	描述
action	McAction	

action_desc	string	
-------------	--------	--

4.3.2.2 运控低层接口

JointCommand

- 协议路径: aimdk_msgs/hal/msg/JointCommand.msg

字段	数据类型	描述
name	string	
position	float64	
velocity	float64	
effort	float64	
stiffness	float64	
damping	float64	

JointState

- 协议路径: aimdk_msgs/hal/msg/JointState.msg

字段	数据类型	描述
name	string	
position	float64	
velocity	float64	
effort	float64	
coil_temp	uint8	
motor_temp	uint8	
motor_vol	uint8	

McLocomotionVelocity

- 协议路径: aimdk_msgs/mc/motion/McLocomotionVelocity.msg

字段	数据类型	描述
header	MessageHeader	
source	string	
forward_velocity	float64	
lateral_velocity	float64	

angular_velocity	float64	
pitch_velocity	float64	
level	float64	

McActionStatus

- 协议路径: aimdk_msgs/mc/action/McActionStatus.msg

字段	数据类型	描述
value	int32	

McActionStatus 枚举字典

常量名	数值	描述
IDLE	0	
RUNNING	100	
TRANSITION	200	

McAction

- 协议路径: aimdk_msgs/mc/action/McAction.msg

字段	数据类型	描述
value	int32	

McAction 枚举字典

常量名	数值	描述
PASSIVE_DEFAULT	1	
DAMPING_DEFAULT	2	
QUADRUPED_TO_BIPED	10	
QUADRUPED_TO_BIPED_ROTATE	11	
QUADRUPED_TO_BIPED_FRONTFLIP	12	
BIPED_TO_QUADRUPED	20	
BIPED_TO_QUADRUPED_ROTATE	21	
BIPED_TO_QUADRUPED_FRONTFLIP	22	
QUADRUPED_RECOVERY	100	

QUADRUPED_STAND_DEFAULT	101	
QUADRUPED_LOCOMOTION_DEFAULT	102	
QUADRUPED_LOCOMOTION_RUN	112	
QUADRUPED_LOCOMOTION_TERRAIN	103	
QUADRUPED_LOCOMOTION_BIONIC	104	
QUADRUPED_LOCOMOTION_FRONTFLIP	105	
QUADRUPED_LOCOMOTION_BACKFLIP	106	
QUADRUPED_LOCOMOTION_HANDSHAKE	107	
QUADRUPED_LOCOMOTION_SIT	108	
QUADRUPED_LOCOMOTION_DANCE	109	
QUADRUPED_GET_DOWN_DEFAULT	110	
QUADRUPED_SIT_DOWN_DEFAULT	111	
QUADRUPED_LOCOMOTION_Pride	113	
QUADRUPED_LOCOMOTION_PLEASURE	114	
QUADRUPED_LOCOMOTION_HABDSHAKE	115	
QUADRUPED_LOCOMOTION_JUMP	116	
BIPED_RECOVERY	200	
BIPED_STAND_DEFAULT	201	
BIPED_LOCOMOTION_DEFAULT	202	
BIPED_LOCOMOTION_TERRAIN	203	
BIPED_LOCOMOTION_ROTATE_LOCAL	204	
BIPED_LOCOMOTION_BACKBEND	205	
BIPED_LOCOMOTION_WBC	300	

4.3.3 交互控制模块

4.3.3.1 语音控制

AudioCapture

- 协议路径: aimdk_msgs/hal/audio/msg/AudioCapture.msg

字段	数据类型	描述
----	------	----

stamps	builtin_interfaces/Time	
mic_channels	uint8	
ref_channels	uint8	
info	AudioInfo	
data	AudioData	
pkg_name	string	
mic_source	uint32	

AudioInfo

- 协议路径: aimdk_msgs/hal/audio/msg/AudioInfo.msg

字段	数据类型	描述
channels	uint8	
sample_rate	uint32	
size	uint32	
sample_format	string	
coding_format	string	

AudioData

- 协议路径: aimdk_msgs/hal/audio/msg/AudioData.msg

字段	数据类型	描述
data	uint8[]	

TtsPriorityLevel

- 协议路径: aimdk_msgs/interaction/msg/TtsPriorityLevel.msg

字段	数据类型	描述
value	uint8	

TtsPriorityLevel 枚举字典

常量名	数值	描述
UNKNOWN	0x00	
BACKGROUND_L1	0x01	

SERVICE_L2	0x02	
MISSION_L4	0x04	
INTERACTION_L6	0x06	
WARNING_L8	0x08	
SAFETY_L10	0x0a	

PlayTtsRequest

- 协议路径: aimdk_msgs/interaction/msg/PlayTtsRequest.msg

字段	数据类型	描述
text	string	
priority_level	TtsPriorityLevel	
priority_weight	uint32	
domain	string	
trace_id	string	
is_interrupted	bool	

PlayTtsResponse

- 协议路径: aimdk_msgs/interaction/msg/PlayTtsResponse.msg

字段	数据类型	描述
text	string	
priority_level	TtsPriorityLevel	
priority_weight	uint32	
domain	string	
trace_id	string	
is_success	bool	
error_message	string	
estimated_duration	uint32	

AudioFile

- 协议路径: aimdk_msgs/hal/audio/msg/AudioFile.msg

字段	数据类型	描述
pkg_name	string	
file_name	string	
file_path	string	
info	AudioInfo	
priority	uint32	
priority_weight	uint32	

4.3.3.2 屏幕控制

4.3.4 硬件抽象模块

4.3.4.1 触觉传感器

TouchState

- 协议路径: aimdk_msgs/hal/msg/TouchState.msg

字段	数据类型	描述
header	MessageHeader	
event_type	uint8	

TouchState 枚举字典

常量名	数值	描述
TOUCH_EVENT_NONE	0	
TOUCH_EVENT_CLICK_SINGLE	1	
TOUCH_EVENT_CLICK_DOUBLE	2	
TOUCH_EVENT_CLICK_TRIPLE	3	
TOUCH_EVENT_CLICK_QUAD	4	
TOUCH_EVENT_CLICK_QUINT	5	
TOUCH_EVENT_CLICK_SEXT	6	
TOUCH_EVENT_CLICK_MULTI	7	
TOUCH_EVENT_SLIDE_LEFT	8	
TOUCH_EVENT_SLIDE_RIGHT	9	

4.3.4.2 视觉传感器

4.3.4.3 电源管理

PmuState

- 协议路径: aimdk_msgs/hal/msg/PmuState.msg

字段	数据类型	描述
pmu_protocol_version	string	协议版本
pmu_hardware_version	string	硬件版本
pmu_software_version	string	软件版本
pmu_bool_status	uint32	
vcc_3v3_voltage	float32	LDO 3v3电压(单位: V)
vcc_1v8_voltage	float32	LDO 1v8电压(单位: V)
vcc_1v2_voltage	float32	LDO 1v2电压(单位: V)
nfc_5v_voltage	float32	NFC 5v电压(单位: V)
soc_5v_voltage	float32	Soc RK3576 5V电压(单位: V)
sys_5v_voltage	float32	系统 5V电压(单位: V)
sys_12v_voltage	float32	系统 12V电压(单位: V)
orin_12v_voltage	float32	Orin 12v电压(单位: V)
ext_12v_voltage	float32	二开 12V电压(单位: V)
ext_24v_voltage	float32	二开 24V电压(单位: V)
arm_48v_voltage	float32	上肢 48V电压(单位: V)
leg_48v_voltage	float32	下肢 48V电压(单位: V)
soc_5v_current	float32	Soc RK3576 5V电流(单位: A)
sys_5v_current	float32	系统 5V电流(单位: A)
sys_12v_current	float32	系统 12V电流(单位: A)
orin_12v_current	float32	Orin 12v电流(单位: A)
ext_12v_current	float32	二开 12V电流(单位: A)
ext_24v_current	float32	二开 24V电流(单位: A)
arm_48v_current	float32	上肢 48V电流(单位: A)

leg_48v_current	float32	下肢 48V电流(单位: A)
bms_manufacturer	string	厂家信息
bms_serial_number	string	序列号
bms_protocol_version	string	协议版本
bms_hardware_version	string	硬件版本
bms_software_version	string	软件版本
bms_status	uint32	状态位和状态码
bms_balance_line_resistance	uint16	电池包均衡线电阻（单位：mΩ）
bms_voltage	uint32	电池包电压（单位：V）
bms_current	int32	电池包电流（单位：A，充电为正，放电为负）
bms_power	int32	电池包输出功率（单位：W）
bms_temperature	uint32	电池包当前温度（单位：℃）
bms_remaining_capacity	uint32	当前电池剩余容量（单位：mAh）
bms_remaining_capacity_percentage	uint8	当前电池剩余容量百分比（单位：%）
bms_cycle_count	uint16	当前电池包完整循环次数（单位：次）
bms_cycle_total_capacity	uint32	总充放电容量累计（单位：Ah）

4.4 示例代码

本模块提供各个功能模块的示例代码，帮助开发者快速理解 ROS 2 节点编写规范及 SDK 接口调用流程。

Cpp代码运行前置条件

1. 环境依赖：需先编译并加载 `aimdk_msgs` 接口依赖包。
2. 源码编译：在 `primebot_sdk/examples/cpp` 下执行 `colcon build --packages-select aimdk_examples_cpp --symlink-install` 编译构建。
3. 加载变量：运行前需执行 `source install/setup.bash` 激活工作空间环境。

Python代码运行前置条件

1. 环境依赖：必须确保包含对 `aimdk_msgs` 依赖消息包的引用支持。
2. 加载变量：运行前需执行 `source install/setup.bash` 激活 ROS 2 通信环境，激活后可直接使用 `python3` 独立载入脚本。

4.3.4.4 灯语控制

4.4.1 运动控制模块

4.4.1.1 运控高层接口

代码示例

动作控制

动作控制 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp set_mc_action`

```
#include "aimdk_msgs/msg/common_response.hpp"
#include "aimdk_msgs/msg/common_request.hpp"
#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/msg/mc_action_command.hpp"
#include "aimdk_msgs/msg/mc_action_status.hpp"
#include "aimdk_msgs/msg/request_header.hpp"
#include "aimdk_msgs/srv/get_mc_action.hpp"
#include "aimdk_msgs/srv/set_mc_action.hpp"
#include "rclcpp/rclcpp.hpp"
#include <chrono>
#include <iostream>
#include <memory>
#include <signal.h>
#include <thread>

std::shared_ptr<rclcpp::Node> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        RCLCPP_INFO(g_node->get_logger(), "Received signal %d, shutting
down...",
                    signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

class SetMcActionClient : public rclcpp::Node {
public:
    SetMcActionClient() : Node("set_mc_action_client") {
        set_client_ = this->create_client<aimdk_msgs::srv::SetMcAction>(
            "/aimdk_5Fmsgs/srv/SetMcAction");
        get_client_ = this->create_client<aimdk_msgs::srv::GetMcAction>(
```

```

        "/aimdk_5Fmsgs/srv/GetMcAction");
RCLCPP_INFO(this->get_logger(), "✅ SetMcAction client node created.");

wait_for_service(set_client_, "/aimdk_5Fmsgs/srv/SetMcAction");
wait_for_service(get_client_, "/aimdk_5Fmsgs/srv/GetMcAction");
}

bool set_action(int32_t action_value) {
    try {
        auto request =
std::make_shared<aimdk_msgs::srv::SetMcAction::Request>();
        request->header.stamp = this->now();
        request->command.action.value = action_value;
        request->command.action_desc = "";

        RCLCPP_INFO(this->get_logger(), "📬 Sending request: action_id=%d",
            action_value);

        auto future = set_client_->async_send_request(request);
        auto retcode = rclcpp::spin_until_future_complete(
            shared_from_this(), future, std::chrono::seconds(2));

        if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
            RCLCPP_ERROR(this->get_logger(), "❌ Service call failed or timed
out.");
            return false;
        }

        auto response = future.get();
        if (response->response.status.value ==
aimdk_msgs::msg::CommonState::SUCCESS) {
            RCLCPP_INFO(this->get_logger(), "✅ SetMcAction request accepted by
service.");
            return true;
        }

        RCLCPP_ERROR(this->get_logger(), "❌ Failed to set robot mode: %s",
            response->response.message.c_str());
        return false;
    } catch (const std::exception &e) {
        RCLCPP_ERROR(this->get_logger(), "Exception occurred: %s", e.what());
        return false;
    }
}

bool wait_for_action(

```

```

    int32_t expected_action_id,
    std::chrono::seconds timeout = std::chrono::seconds(10),
    std::chrono::milliseconds poll_interval =
std::chrono::milliseconds(200)) {
    auto deadline = std::chrono::steady_clock::now() + timeout;

    RCLCPP_INFO(this->get_logger(),
        "⌚ Waiting for target action_id=%d to reach RUNNING
state...",
        expected_action_id);

    while (rclcpp::ok() && std::chrono::steady_clock::now() < deadline) {
        int32_t current_action_id = 0;
        int32_t current_status = aimdk_msgs::msg::McActionStatus::IDLE;
        if (!get_action_status(current_action_id, current_status)) {
            std::this_thread::sleep_for(poll_interval);
            continue;
        }

        if (current_status == aimdk_msgs::msg::McActionStatus::RUNNING &&
            current_action_id == expected_action_id) {
            RCLCPP_INFO(this->get_logger(),
                "✅ Target action reached and is running:
action_id=%d",
                expected_action_id);
            return true;
        }

        std::this_thread::sleep_for(poll_interval);
    }

    RCLCPP_ERROR(this->get_logger(),
        "❌ Timed out waiting for target action_id=%d to reach "
        "RUNNING state.",
        expected_action_id);
    return false;
}

private:
bool get_action_status(int32_t &action_id, int32_t &status) {
    try {
        auto request =
std::make_shared<aimdk_msgs::srv::GetMcAction::Request>();
        request->request = aimdk_msgs::msg::CommonRequest();
        request->request.header.stamp = this->now();
    }

```

```

    auto future = get_client_>async_send_request(request);
    auto retcode = rclcpp::spin_until_future_complete(
        shared_from_this(), future, std::chrono::seconds(2));

    if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
        RCLCPP_WARN(this->get_logger(),
            "⚠ Get current action request service call failed or
timed "
                "out.");
        return false;
    }

    auto response = future.get();
    action_id = response->info.current_action.value;
    status = response->info.status.value;
    return true;
} catch (const std::exception &e) {
    RCLCPP_ERROR(this->get_logger(), "Exception occurred: %s", e.what());
    return false;
}
}

template <typename ClientT>
void wait_for_service(
    const std::shared_ptr<ClientT> &client,
    const std::string &service_name) {
    while (!client->wait_for_service(std::chrono::seconds(2))) {
        if (!rclcpp::ok()) {
            return;
        }
        RCLCPP_INFO(this->get_logger(), "⌚ Service unavailable, waiting:
%s",
            service_name.c_str());
    }
    RCLCPP_INFO(this->get_logger(), "🟢 Service available: %s",
        service_name.c_str());
}

rclcpp::Client<aimdk_msgs::srv::SetMcAction>::SharedPtr set_client_;
rclcpp::Client<aimdk_msgs::srv::GetMcAction>::SharedPtr get_client_;
};

int main(int argc, char *argv[]) {
    try {
        rclcpp::init(argc, argv);

```

```

signal(SIGINT, signal_handler);
signal(SIGTERM, signal_handler);

int32_t action_id = 0;
std::cout << "Enter action_id: ";
std::cin >> action_id;

g_node = std::make_shared<SetMcActionClient>();
auto client = std::dynamic_pointer_cast<SetMcActionClient>(g_node);
if (client) {
    if (!client->set_action(action_id)) {
        g_node.reset();
        rclcpp::shutdown();
        return 1;
    }
    if (!client->wait_for_action(action_id)) {
        g_node.reset();
        rclcpp::shutdown();
        return 1;
    }
}

g_node.reset();
rclcpp::shutdown();

return 0;
} catch (const std::exception &e) {
    RCLCPP_ERROR(rclcpp::get_logger("main"),
        "Program exited with exception: %s", e.what());
    return 1;
}
}

```

动作控制 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/set_mc_action.py`

```

#!/usr/bin/env python3

import time

import rclpy
import rclpy.logging
from rclpy.node import Node

```

```

from aimdk_msgs.msg import (
    CommonRequest,
    CommonState,
    McAction,
    McActionCommand,
    McActionStatus,
    RequestHeader,
)
from aimdk_msgs.srv import GetMcAction, SetMcAction

class SetMcActionClient(Node):
    def __init__(self):
        super().__init__('set_mc_action_client')
        self.set_client = self.create_client(
            SetMcAction, '/aimdk_5Fmsgs/srv/SetMcAction'
        )
        self.get_client = self.create_client(
            GetMcAction, '/aimdk_5Fmsgs/srv/GetMcAction'
        )
        self.get_logger().info('✅ SetMcAction client node created.')

        self.wait_for_service(self.set_client,
                               '/aimdk_5Fmsgs/srv/SetMcAction')
        self.wait_for_service(self.get_client,
                               '/aimdk_5Fmsgs/srv/GetMcAction')

    def wait_for_service(self, client, service_name: str):
        while not client.wait_for_service(timeout_sec=2.0):
            if not rclpy.ok():
                return
            self.get_logger().info(f'⌚ Service unavailable, waiting: {service_name}')
        self.get_logger().info(f'🟢 Service available: {service_name}')

    def set_action(self, action_id: int) -> bool:
        try:
            request = SetMcAction.Request()
            request.header = RequestHeader()

            command = McActionCommand()
            command.action = McAction()
            command.action.value = action_id
            command.action_desc = ''
            request.command = command

```

```

        self.get_logger().info(f'📬 Sending request: action_id={
{action_id}}')
        request.header.stamp = self.get_clock().now().to_msg()
        future = self.set_client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)

        if not future.done():
            self.get_logger().error('❌ Service call failed or timed
out.')
```

```

            return False

        response = future.result()
        if response is None:
            self.get_logger().error('❌ Service call failed or timed
out.')
```

```

            return False

        if response.response.status.value == CommonState.SUCCESS:
            self.get_logger().info('✅ SetMcAction request accepted by
service.')
```

```

            return True

        self.get_logger().error(
            f'❌ Failed to set robot mode: {response.response.message}'
        )
        return False
    except Exception as e:
        self.get_logger().error(f'Exception occurred: {e}')
        return False

def wait_for_action(
    self,
    expected_action_id: int,
    timeout_sec: float = 15.0,
    poll_interval_sec: float = 0.2,
) -> bool:
    deadline = time.monotonic() + timeout_sec

    self.get_logger().info(
        f'🕒 Waiting for target action_id={expected_action_id} to reach
RUNNING state...'
    )

    while rclpy.ok() and time.monotonic() < deadline:
        action_id, status = self.get_action_status()
        if action_id is None or status is None:

```

```

        time.sleep(poll_interval_sec)
        continue

    if (
        action_id == expected_action_id
        and status == McActionStatus.RUNNING
    ):
        self.get_logger().info(
            f'✅ Target action reached and is running: action_id={
expected_action_id}'
        )
        return True

    time.sleep(poll_interval_sec)

    self.get_logger().error(
        f'❌ Timed out waiting for target action_id=
expected_action_id' '
        'to reach RUNNING state.'
    )
    return False

def get_action_status(self):
    try:
        request = GetMcAction.Request()
        request.request = CommonRequest()

        request.request.header.stamp = self.get_clock().now().to_msg()
        future = self.get_client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)

        if not future.done():
            self.get_logger().warning(
                '⚠️ Get current action request service call failed or
timed out.'
            )
            return None, None

        response = future.result()
        if response is None:
            self.get_logger().warning(
                '⚠️ Get current action request service call failed or
timed out.'
            )
            return None, None

```

```

        return response.info.current_action.value,
response.info.status.value
    except Exception as e:
        self.get_logger().error(f'Exception occurred: {e}')
        return None, None

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        action_id = 0
        print('Enter action_id: ', end='')
        try:
            action_id = int(input())
        except Exception:
            pass

        node = SetMcActionClient()
        if not node.set_action(action_id):
            node.destroy_node()
            node = None
            if rclpy.ok():
                rclpy.shutdown()
            return 1

        if not node.wait_for_action(action_id):
            node.destroy_node()
            node = None
            if rclpy.ok():
                rclpy.shutdown()
            return 1

        node.destroy_node()
        node = None
        if rclpy.ok():
            rclpy.shutdown()
        return 0
    except KeyboardInterrupt:
        if node is not None:
            node.destroy_node()
            node = None
        if rclpy.ok():
            rclpy.shutdown()
        return 0
    except Exception as e:

```

```

    rclpy.logging.get_logger('main').error(
        f'Program exited with exception: {e}'
    )
    if node is not None:
        node.destroy_node()
        node = None
    if rclpy.ok():
        rclpy.shutdown()
    return 1

if __name__ == '__main__':
    raise SystemExit(main())

```

代码示例

速度控制

速度控制 Cpp示例

执行前置： 需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp mc_locomotion_velocity`

```

#include "aimdk_msgs/msg/mc_locomotion_velocity.hpp"
#include "aimdk_msgs/msg/common_request.hpp"
#include "aimdk_msgs/msg/common_response.hpp"
#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/msg/common_task_response.hpp"
#include "aimdk_msgs/msg/mc_input_action.hpp"
#include "aimdk_msgs/msg/message_header.hpp"
#include "aimdk_msgs/srv/get_current_input_source.hpp"
#include "aimdk_msgs/srv/set_mc_input_source.hpp"

#include "rclcpp/rclcpp.hpp"
#include <cmath>
#include <chrono>
#include <iostream>
#include <memory>
#include <signal.h>
#include <thread>

class DirectVelocityControl : public rclcpp::Node {
public:
    DirectVelocityControl() : Node("direct_velocity_control") {
        // Create publisher
        publisher_ = this->
>create_publisher<aimdk_msgs::msg::McLocomotionVelocity>(

```

```

        "/aima/mc/locomotion/velocity", 10);
// Create service clients
set_client_ = this->create_client<aimdk_msgs::srv::SetMcInputSource>(
    "/aimdk_5Fmsgs/srv/SetMcInputSource");
get_client_ = this->create_client<aimdk_msgs::srv::GetCurrentInputSource>(
    "/aimdk_5Fmsgs/srv/GetCurrentInputSource");

// Maximum speed limits
max_forward_speed_ = 1.0; // m/s
max_lateral_speed_ = 1.0; // m/s
max_angular_speed_ = 1.0; // rad/s
// Minimum speed limits (0 is also OK)
min_forward_speed_ = 0.2; // m/s
min_lateral_speed_ = 0.2; // m/s
min_angular_speed_ = 0.1; // rad/s

RCLCPP_INFO(this->get_logger(), "🟢 Direct velocity control node
started.");
}

void start_publish() {
    if (timer_ != nullptr) {
        return;
    }
    // Set timer to periodically publish velocity messages (50Hz)
    timer_ = this->create_wall_timer(
        std::chrono::milliseconds(20),
        std::bind(&DirectVelocityControl::publish_velocity, this));
}

bool register_input_source() {
    if (!wait_for_service(set_client_,
        "/aimdk_5Fmsgs/srv/SetMcInputSource")) {
        return false;
    }
}

auto request =
    std::make_shared<aimdk_msgs::srv::SetMcInputSource::Request>();
request->action.value = 1001;
request->input_source.name = "node";
request->input_source.priority = 80;
request->input_source.timeout = 1000;

auto timeout = std::chrono::milliseconds(2000);

```

```

request->request.header.stamp = this->now();
auto future = set_client_->async_send_request(request);
auto retcode = rclcpp::spin_until_future_complete(
    this->shared_from_this(), future, timeout);
if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
    RCLCPP_ERROR(this->get_logger(), "❌ SetMcInputSource failed or timed
out");
    return false;
}

auto response = future.get();
int state = response->response.state.value;
RCLCPP_INFO(this->get_logger(),
    "✅ Set input source succeeded: state=%d, task_id=%lu",
state,
    response->response.task_id);
return true;
}

bool get_current_input_source() {
    if (!wait_for_service(get_client_,
"/aimdk_5Fmsgs/srv/GetCurrentInputSource")) {
        return false;
    }

    RCLCPP_INFO(this->get_logger(), "⌚ Querying current input source");

    auto request =
        std::make_shared<aimdk_msgs::srv::GetCurrentInputSource::Request>
();
    request->request = aimdk_msgs::msg::CommonRequest();
    request->request.header.stamp = this->now();

    auto timeout = std::chrono::milliseconds(2000);
    auto future = get_client_->async_send_request(request);
    auto retcode = rclcpp::spin_until_future_complete(
        this->shared_from_this(), future, timeout);
    if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
        RCLCPP_WARN(this->get_logger(), "❌ GetCurrentInputSource timed
out");
        return false;
    }

    auto response = future.get();
    if (response->response.header.code == 0) {
        RCLCPP_INFO(this->get_logger(),

```

```

        "🟢 Current input source: name=%s, priority=%d,
timeout=%d",
        response->input_source.name.c_str(),
        response->input_source.priority,
        response->input_source.timeout);
    return true;
}

    RCLCPP_WARN(this->get_logger(), "❌ GetCurrentInputSource returned
code=%ld",
        response->response.header.code);
    return false;
}

void publish_velocity() {
    auto msg = std::make_unique<aimdk_msgs::msg::McLocomotionVelocity>();
    msg->header = aimdk_msgs::msg::MessageHeader();
    msg->header.stamp = this->now();
    msg->source = "node"; // Set message source
    msg->forward_velocity = forward_velocity_;
    msg->lateral_velocity = lateral_velocity_;
    msg->angular_velocity = angular_velocity_;
    msg->pitch_velocity = 0.0;
    msg->level = 0.0;

    publisher_->publish(std::move(msg));
}

void clear_velocity() {
    forward_velocity_ = 0.0;
    lateral_velocity_ = 0.0;
    angular_velocity_ = 0.0;
}

bool set_forward(double forward) {
    if (std::abs(forward) < 0.005) {
        forward_velocity_ = 0.0;
        return true;
    } else if ((std::abs(forward) > max_forward_speed_) ||
        (std::abs(forward) < min_forward_speed_)) {
        RCLCPP_ERROR(this->get_logger(), "❌ input value out of range,
exiting");
        return false;
    } else {
        forward_velocity_ = forward;
        return true;
    }
}

```

```

    }
}

bool set_lateral(double lateral) {
    if (std::abs(lateral) < 0.005) {
        lateral_velocity_ = 0.0;
        return true;
    } else if ((std::abs(lateral) > max_lateral_speed_) ||
               (std::abs(lateral) < min_lateral_speed_)) {
        RCLCPP_ERROR(this->get_logger(), "❌ input value out of range,
exiting");
        return false;
    } else {
        lateral_velocity_ = lateral;
        return true;
    }
}

bool setAngular(double angular) {
    if (std::abs(angular) < 0.005) {
        angular_velocity_ = 0.0;
        return true;
    } else if ((std::abs(angular) > max_angular_speed_) ||
               (std::abs(angular) < min_angular_speed_)) {
        RCLCPP_ERROR(this->get_logger(), "❌ input value out of range,
exiting");
        return false;
    } else {
        angular_velocity_ = angular;
        return true;
    }
}

private:
    template <typename ClientT>
    bool wait_for_service(const std::shared_ptr<ClientT> &client,
                        const char *service_name) {
        while (!client->wait_for_service(std::chrono::seconds(2))) {
            if (!rclcpp::ok()) {
                return false;
            }
            RCLCPP_INFO(this->get_logger(), "⌚ Waiting for service: %s",
service_name);
        }
        return true;
    }
}

```

```

    rclcpp::Publisher<aimdk_msgs::msg::McLocomotionVelocity>::SharedPtr
        publisher_;
    rclcpp::Client<aimdk_msgs::srv::SetMcInputSource>::SharedPtr set_client_;
    rclcpp::Client<aimdk_msgs::srv::GetCurrentInputSource>::SharedPtr
get_client_;
    rclcpp::TimerBase::SharedPtr timer_;

    double forward_velocity_;
    double lateral_velocity_;
    double angular_velocity_;

    double max_forward_speed_;
    double max_lateral_speed_;
    double max_angular_speed_;

    double min_forward_speed_;
    double min_lateral_speed_;
    double min_angular_speed_;
};

std::shared_ptr<DirectVelocityControl> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        g_node->clear_velocity();
        RCLCPP_INFO(g_node->get_logger(),
            "⌚ Received signal %d, clearing velocity and shutting
down...",
                signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

int main(int argc, char *argv[]) {
    rclcpp::init(argc, argv);
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    g_node = std::make_shared<DirectVelocityControl>();
    auto node = g_node;

    node->get_current_input_source();

```

```

if (!node->register_input_source()) {
    RCLCPP_ERROR(node->get_logger(),
        "❌ Input source registration failed, exiting");
    g_node.reset();
    rclcpp::shutdown();
    return 1;
}

// get and check control values
// notice that mc has thresholds to start movement
double forward, lateral, angular;
std::cout << "Enter forward speed 0 or ±(0.2 ~ 1.0) m/s: ";
std::cin >> forward;
if (!node->set_forward(forward)) {
    return 2;
}
std::cout << "Enter lateral speed 0 or ±(0.2 ~ 1.0) m/s: ";
std::cin >> lateral;
if (!node->set_lateral(lateral)) {
    return 2;
}
std::cout << "Enter angular speed 0 or ±(0.1 ~ 1.0) rad/s: ";
std::cin >> angular;
if (!node->set_angular(angular)) {
    return 2;
}

RCLCPP_INFO(node->get_logger(),
    "🟢 Start publishing velocity for 5 seconds: Forward %.2f
m/s, "
    "Lateral %.2f m/s, Angular %.2f rad/s",
    forward, lateral, angular);

node->start_publish();

auto start_time = node->now();
bool queried_after_publish = false;
while ((node->now() - start_time).seconds() < 5.0) {
    if (!queried_after_publish &&
        (node->now() - start_time).seconds() > 1.0) {
        node->get_current_input_source();
        queried_after_publish = true;
    }
    rclcpp::spin_some(node);
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

```

```

node->clear_velocity();
node->publish_velocity();
RCLCPP_INFO(node->get_logger(), "✅ 5 seconds elapsed; robot stopped");

g_node.reset();
rclcpp::shutdown();
return 0;
}

```

速度控制 Python示例

执行前置： 需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/mc_locomotion_velocity.py`

```

#!/usr/bin/env python3

import time

import rclpy
from rclpy.node import Node

from aimdk_msgs.msg import CommonRequest, McLocomotionVelocity,
MessageHeader
from aimdk_msgs.srv import GetCurrentInputSource, SetMcInputSource

class DirectVelocityControl(Node):
    def __init__(self):
        super().__init__("direct_velocity_control")

        self.publisher = self.create_publisher(
            McLocomotionVelocity, "/aima/mc/locomotion/velocity", 10
        )
        self.set_client = self.create_client(
            SetMcInputSource, "/aimdk_5Fmsgs/srv/SetMcInputSource"
        )
        self.get_client = self.create_client(
            GetCurrentInputSource,
            "/aimdk_5Fmsgs/srv/GetCurrentInputSource"
        )

        self.forward_velocity = 0.0
        self.lateral_velocity = 0.0
        self.angular_velocity = 0.0

```

```

self.max_forward_speed = 1.0
self.max_lateral_speed = 1.0
self.max_angular_speed = 1.0

self.min_forward_speed = 0.2
self.min_lateral_speed = 0.2
self.min_angular_speed = 0.1

self.timer = None

self.get_logger().info("Direct velocity control node started.")

def wait_for_service(self, client, service_name: str) -> bool:
    while not client.wait_for_service(timeout_sec=2.0):
        if not rclpy.ok():
            return False
        self.get_logger().info(f"Waiting for service: {service_name}")
    return True

def start_publish(self):
    if self.timer is None:
        self.timer = self.create_timer(0.02, self.publish_velocity)

def register_input_source(self) -> bool:
    if not self.wait_for_service(
        self.set_client, "/aimdk_5Fmsgs/srv/SetMcInputSource"
    ):
        return False

    request = SetMcInputSource.Request()
    request.action.value = 1001
    request.input_source.name = "node"
    request.input_source.priority = 80
    request.input_source.timeout = 1000
    request.request.header.stamp = self.get_clock().now().to_msg()

    future = self.set_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)

    if not future.done():
        self.get_logger().error("SetMcInputSource failed or timed out")
        return False

    try:
        response = future.result()
    except Exception as exc:

```

```

        self.get_logger().error(f"SetMcInputSource failed: {exc}")
        return False

    state = response.response.state.value
    self.get_logger().info(
        "Set input source succeeded: "
        f"state={state}, task_id={response.response.task_id}"
    )
    return True

def get_current_input_source(self) -> bool:
    if not self.wait_for_service(
        self.get_client, "/aimdk_5Fmsgs/srv/GetCurrentInputSource"
    ):
        return False

    self.get_logger().info("Querying current input source")

    request = GetCurrentInputSource.Request()
    request.request = CommonRequest()
    request.request.header.stamp = self.get_clock().now().to_msg()

    future = self.get_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)

    if not future.done():
        self.get_logger().warning("GetCurrentInputSource timed out")
        return False

    try:
        response = future.result()
    except Exception as exc:
        self.get_logger().warning(f"GetCurrentInputSource failed:
{exc}")

        return False

    ret_code = response.response.header.code
    if ret_code == 0:
        self.get_logger().info(
            "Current input source: "
            f"name={response.input_source.name}, "
            f"priority={response.input_source.priority}, "
            f"timeout={response.input_source.timeout}"
        )
        return True

```

```

self.get_logger().warning(
    f"GetCurrentInputSource returned code={ret_code}"
)
return False

def publish_velocity(self):
    msg = McLocomotionVelocity()
    msg.header = MessageHeader()
    msg.header.stamp = self.get_clock().now().to_msg()
    msg.source = "node"
    msg.forward_velocity = self.forward_velocity
    msg.lateral_velocity = self.lateral_velocity
    msg.angular_velocity = self.angular_velocity
    msg.pitch_velocity = 0.0
    msg.level = 0.0

    self.publisher.publish(msg)

def clear_velocity(self):
    self.forward_velocity = 0.0
    self.lateral_velocity = 0.0
    self.angular_velocity = 0.0

def set_forward(self, forward: float) -> bool:
    if abs(forward) < 0.005:
        self.forward_velocity = 0.0
        return True
    if abs(forward) > self.max_forward_speed or abs(forward) <
self.min_forward_speed:
        self.get_logger().error("Input forward value out of range,
exiting")
        return False
    self.forward_velocity = forward
    return True

def set_lateral(self, lateral: float) -> bool:
    if abs(lateral) < 0.005:
        self.lateral_velocity = 0.0
        return True
    if abs(lateral) > self.max_lateral_speed or abs(lateral) <
self.min_lateral_speed:
        self.get_logger().error("Input lateral value out of range,
exiting")
        return False
    self.lateral_velocity = lateral
    return True

```

```

def set_angular(self, angular: float) -> bool:
    if abs(angular) < 0.005:
        self.angular_velocity = 0.0
        return True
    if abs(angular) > self.max_angular_speed or abs(angular) <
self.min_angular_speed:
        self.get_logger().error("Input angular value out of range,
exiting")
        return False
    self.angular_velocity = angular
    return True

def main(args=None):
    rclpy.init(args=args)
    node = DirectVelocityControl()

    try:
        node.get_current_input_source()

        if not node.register_input_source():
            node.get_logger().error("Input source registration failed,
exiting")
            return 1

        try:
            forward = float(input("Enter forward speed 0 or +/- (0.2 ~ 1.0)
m/s: "))
            lateral = float(input("Enter lateral speed 0 or +/- (0.2 ~ 1.0)
m/s: "))
            angular = float(input("Enter angular speed 0 or +/- (0.1 ~ 1.0)
rad/s: "))
        except ValueError as exc:
            node.get_logger().error(f"Invalid input: {exc}")
            return 2

        if not node.set_forward(forward):
            return 2
        if not node.set_lateral(lateral):
            return 2
        if not node.set_angular(angular):
            return 2

        node.get_logger().info(
            "Start publishing velocity for 5 seconds: "

```

```

        f"Forward {forward:.2f} m/s, "
        f"Lateral {lateral:.2f} m/s, "
        f"Angular {angular:.2f} rad/s"
    )

    node.start_publish()

    start_time = node.get_clock().now()
    queried_after_publish = False
    while (node.get_clock().now() - start_time).nanoseconds / 1e9 <
5.0:
        elapsed = (node.get_clock().now() - start_time).nanoseconds /
1e9

        if not queried_after_publish and elapsed > 1.0:
            node.get_current_input_source()
            queried_after_publish = True

        rclpy.spin_once(node, timeout_sec=0.1)
        time.sleep(0.001)

    node.clear_velocity()
    node.publish_velocity()
    node.get_logger().info("5 seconds elapsed; robot stopped")
    return 0
finally:
    if node.timer is not None:
        node.timer.cancel()
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

代码示例

预设动作设置

预设动作设置 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp set_preset_motion`

```

#include "aimdk_msgs/msg/common_response.hpp"
#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/msg/common_task_response.hpp"

```

```

#include "aimdk_msgs/msg/mc_preset_motion.hpp"
#include "aimdk_msgs/msg/request_header.hpp"
#include "aimdk_msgs/srv/set_mc_preset_motion.hpp"
#include "rclcpp/rclcpp.hpp"
#include <chrono>
#include <memory>
#include <signal.h>

std::shared_ptr<rclcpp::Node> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        RCLCPP_INFO(g_node->get_logger(), "Received signal %d, shutting
down...",
                    signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

class PresetMotionClient : public rclcpp::Node {
public:
    PresetMotionClient() : Node("preset_motion_client") {
        const std::chrono::seconds timeout(8);

        client_ = this->create_client<aimdk_msgs::srv::SetMcPresetMotion>(
            "/aimdk_5Fmsgs/srv/SetMcPresetMotion");

        RCLCPP_INFO(this->get_logger(), "✅ SetMcPresetMotion client node
created.");

        // Wait for the service to become available
        while (!client_->wait_for_service(std::chrono::seconds(2))) {
            if (!rclcpp::ok()) {
                return;
            }
            RCLCPP_INFO(this->get_logger(), "⌚ Service unavailable,
waiting...");
        }
        RCLCPP_INFO(this->get_logger(),
                    "🟢 Service available, ready to send request.");
    }

    bool send_request(int motion_id) {
        try {

```

```

auto request =
    std::make_shared<aimdk_msgs::srv::SetMcPresetMotion::Request>();
request->header = aimdk_msgs::msg::RequestHeader();

aimdk_msgs::msg::McPresetMotion motion;
motion.value = motion_id; // Preset motion ID
request->motion = motion;
request->interrupt = true; // Interrupt current motion

RCLCPP_INFO(this->get_logger(),
    "📬 Sending request to set preset motion: motion=%d",
    motion_id);

const std::chrono::milliseconds timeout(2000);
request->header.stamp = this->now();
auto future = client_->async_send_request(request);
auto retcode = rclcpp::spin_until_future_complete(shared_from_this(),
    future, timeout);

if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
    RCLCPP_ERROR(this->get_logger(), "❌ Service call failed or timed
out.");
    return false;
}

auto response = future.get();
auto code = response->response.header.code;
auto state = response->response.state.value;

if (code != 0) {
    RCLCPP_WARN(this->get_logger(),
        "❌ Failed to set preset motion: code=%ld, state=%d,
task_id=%lu",
        code, state, response->response.task_id);
    return false;
}

if (state == aimdk_msgs::msg::CommonState::SUCCESS) {
    RCLCPP_INFO(this->get_logger(), "✅ Preset motion set successfully:
%lu",
        response->response.task_id);
    return true;
} else if (state == aimdk_msgs::msg::CommonState::RUNNING) {
    RCLCPP_INFO(this->get_logger(), "⌚ Preset motion executing: %lu",
        response->response.task_id);
    return true;
} else {

```

```

        RCLCPP_WARN(this->get_logger(),
                    "❌ Failed to set preset motion: code=%ld, state=%d,
task_id=%lu",
                    code, state, response->response.task_id);
        return false;
    }
} catch (const std::exception &e) {
    RCLCPP_ERROR(this->get_logger(), "Exception occurred: %s", e.what());
    return false;
}
}

private:
    rclcpp::Client<aimdk_msgs::srv::SetMcPresetMotion>::SharedPtr client_;
};

int main(int argc, char *argv[]) {
    try {
        rclcpp::init(argc, argv);
        signal(SIGINT, signal_handler);
        signal(SIGTERM, signal_handler);

        g_node = std::make_shared<PresetMotionClient>();
        auto client = std::dynamic_pointer_cast<PresetMotionClient>(g_node);

        int motion = 1003;
        std::cout << "TODO: Change input validation for motion ID
        << "Enter preset motion ID (1001-raise, 1002-wave, 1003-handshake,
        "
        "1004-airkiss): ";
        std::cin >> motion;
        if (client) {
            client->send_request(motion);
        }
        g_node.reset();
        rclcpp::shutdown();

        return 0;
    } catch (const std::exception &e) {
        RCLCPP_ERROR(rclcpp::get_logger("main"),
                    "Program exited with exception: %s", e.what());
        return 1;
    }
}

```

预设动作设置 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/set_preset_motion.py`

```
#!/usr/bin/env python3

import rclpy
import rclpy.logging
from rclpy.node import Node

from aimdk_msgs.srv import SetMcPresetMotion
from aimdk_msgs.msg import McPresetMotion, RequestHeader, CommonState

class SetMcPresetMotionClient(Node):
    def __init__(self):
        super().__init__('preset_motion_client')
        self.client = self.create_client(
            SetMcPresetMotion, '/aimdk_5Fmsgs/srv/SetMcPresetMotion')
        self.get_logger().info('✅ SetMcPresetMotion client node created.')

        # Wait for the service to become available
        while not self.client.wait_for_service(timeout_sec=2.0):
            self.get_logger().info('⌚ Service unavailable, waiting...')

        self.get_logger().info('🟢 Service available, ready to send request.')

    def send_request(self, motion_id: int) -> bool:
        request = SetMcPresetMotion.Request()
        request.header = RequestHeader()

        motion = McPresetMotion()

        motion.value = motion_id

        request.motion = motion
        request.interrupt = True

        self.get_logger().info(
            f'📬 Sending request to set preset motion: motion={motion_id}')

        request.header.stamp = self.get_clock().now().to_msg()
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)
```

```

    if not future.done():
        self.get_logger().error('❌ Service call failed or timed out.')
        return False

    response = future.result()
    if response is None:
        self.get_logger().error('❌ Service call failed or timed out.')
        return False

    code = response.response.header.code
    state = response.response.state.value

    if code != 0:
        self.get_logger().warning(
            f'❌ Failed to set preset motion: code={code}, state={state}, task_id={response.response.task_id}')
        return False

    if state == CommonState.SUCCESS:
        self.get_logger().info(
            f'✅ Preset motion set successfully: {response.response.task_id}')
        return True
    elif state == CommonState.RUNNING:
        self.get_logger().info(
            f'⌚ Preset motion executing: {response.response.task_id}')
        return True
    else:
        self.get_logger().warning(
            f'❌ Failed to set preset motion: code={code}, state={state}, task_id={response.response.task_id}')
        return False

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        # TODO: Change input validation for motion ID
        motion = int(input(
            "Enter preset motion ID (1001-raise, 1002-wave, 1003-handshake, 1004-airkiss): "))

        node = SetMcPresetMotionClient()
        node.send_request(motion)
    except KeyboardInterrupt:
        pass

```

```

except Exception as e:
    rclpy.logging.get_logger('main').error(
        f'Program exited with exception: {e}')

if node:
    node.destroy_node()
if rclpy.ok():
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

4.4.2 交互控制模块

4.4.2.1 语音控制

代码示例

音频播放

音频播放 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp play_audio`

```

/**
 * @brief Example client for /aimdk_5Fmsgs/srv/PlayAudioFile
 *
 * The following ROS parameters can be set via startup arguments:
 * --ros-args -p <name>:=<value>
 *
 * Supported parameters:
 * - file_name: audio file name only
 * - file_path: directory containing the audio file
 *
 * Examples:
 * ros2 run aimdk_examples_cpp play_audio --ros-args -p
 *   file_name:=demo.wav -p file_path:=/tmp
 *
 * ros2 run aimdk_examples_cpp play_audio --ros-args -p
 *   file_name:=撒娇.wav -p file_path:=/agibot/software/interaction/bin/cfg
 *
 * Other request fields use built-in defaults and are not configurable from
 * the command line in this demo.
 */
#include "aimdk_msgs/msg/common_request.hpp"

```

```

#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/srv/play_audio_file.hpp"
#include "rclcpp/rclcpp.hpp"

#include <chrono>
#include <stdint>
#include <stdlib.h>
#include <exception>
#include <memory>
#include <signal.h>
#include <string>

std::shared_ptr<rclcpp::Node> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        RCLCPP_INFO(g_node->get_logger(), "Received signal %d, shutting
down...",
                    signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

class PlayAudioFileClient : public rclcpp::Node {
public:
    PlayAudioFileClient() : Node("play_audio_file_client") {
        file_name_ =
            this->declare_parameter<std::string>("file_name", file_name_);
        file_path_ =
            this->declare_parameter<std::string>("file_path", file_path_);

        client_ = this->create_client<aimdk_msgs::srv::PlayAudioFile>
(service_name_);
        RCLCPP_INFO(this->get_logger(), "✅ PlayAudioFile client node
created.");
    }

    bool send_request() {
        if (!wait_for_service()) {
            return false;
        }

        auto request =
std::make_shared<aimdk_msgs::srv::PlayAudioFile::Request>();

```

```

request->request = aimdk_msgs::msg::CommonRequest();
request->request.header.stamp = this->now();

request->file.pkg_name = pkg_name_;
request->file.file_name = file_name_;
request->file.file_path = file_path_;

if (request->file.file_name.empty()) {
    RCLCPP_ERROR(this->get_logger(), "❌ file_name is empty.");
    return false;
}

request->file.info.channels = static_cast<uint8_t>(channels_);
request->file.info.sample_rate = static_cast<uint32_t>(sample_rate_);
request->file.info.size = static_cast<uint32_t>(size_);
request->file.info.sample_format = sample_format_;
request->file.info.coding_format = coding_format_;
request->file.priority = static_cast<uint32_t>(priority_);
request->file.priority_weight = static_cast<uint32_t>
(priority_weight_);

RCLCPP_INFO(this->get_logger(),
    "📁 Sending PlayAudioFile request: file=%s, path=%s,
pkg=%s, "
    "ch=%d, sr=%d, fmt=%s, coding=%s, priority=%d, weight=%d",
    request->file.file_name.c_str(), request-
>file.file_path.c_str(),
    request->file.pkg_name.c_str(), channels_, sample_rate_,
    sample_format_.c_str(), coding_format_.c_str(), priority_,
    priority_weight_);

auto future = client_->async_send_request(request);
auto retcode = rclcpp::spin_until_future_complete(shared_from_this()),
future,

std::chrono::seconds(5));
if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
    RCLCPP_ERROR(this->get_logger(),
        "❌ PlayAudioFile call failed or timed out.");
    return false;
}

const auto response = future.get();
const auto code = response->reponse.header.code;
const auto status = response->reponse.status.value;
const bool ok =

```

```

        code == 0 || status == aimdk_msgs::msg::CommonState::SUCCESS;

    if (ok) {
        RCLCPP_INFO(this->get_logger(),
            "✅ PlayAudioFile accepted. code=%ld status=%d msg=%s",
code,
            status, response->reponse.message.c_str());
        return true;
    }

    RCLCPP_ERROR(this->get_logger(), "❌ PlayAudioFile rejected. code=%ld
status=%d "
                    "msg=%s",
            code, status, response->reponse.message.c_str());
    return false;
}

private:
    bool wait_for_service() {
        while (!client_->wait_for_service(std::chrono::seconds(2))) {
            if (!rclcpp::ok()) {
                return false;
            }
            RCLCPP_INFO(this->get_logger(), "⌚ Waiting for service: %s",
                service_name_.c_str());
        }
        RCLCPP_INFO(this->get_logger(), "🟢 Service available, ready to send
request.");
        return true;
    }

    std::string service_name_ = "/aimdk_5Fmsgs/srv/PlayAudioFile";
    std::string pkg_name_ = "sdk_demo";
    std::string file_name_ = "撒娇.wav";
    std::string file_path_ = "/agibot/software/interaction/bin/cfg";
    std::string sample_format_ = "S16_LE";
    std::string coding_format_ = "wave";
    int channels_ = 1;
    int sample_rate_ = 16000;
    int size_ = 0;
    int priority_ = 6;
    int priority_weight_ = 0;
    rclcpp::Client<aimdk_msgs::srv::PlayAudioFile>::SharedPtr client_;
};

int main(int argc, char *argv[]) {

```

```

try {
    rclcpp::init(argc, argv);
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    g_node = std::make_shared<PlayAudioFileClient>();
    auto client = std::dynamic_pointer_cast<PlayAudioFileClient>(g_node);
    const bool ok = client ? client->send_request() : false;

    g_node.reset();
    rclcpp::shutdown();
    return ok ? 0 : 1;
} catch (const std::exception &e) {
    RCLCPP_ERROR(rclcpp::get_logger("main"),
        "❌ Program exited with exception: %s", e.what());
    return 1;
}
}

```

音频播放 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/play_audio.py`

```

#!/usr/bin/env python3

import rclpy
import rclpy.logging
from rclpy.node import Node

from aimdk_msgs.msg import CommonRequest, CommonState
from aimdk_msgs.srv import PlayAudioFile

class PlayAudioFileClient(Node):
    def __init__(self):
        super().__init__("play_audio_file_client")
        self.file_name = self.declare_parameter("file_name", "撒
娇.wav").value
        self.file_path = self.declare_parameter("file_path",
            "/agibot/software/interaction/bin/cfg").value

        self.service_name = "/aimdk_5Fmsgs/srv/PlayAudioFile"
        self.pkg_name = "sdk_demo"
        self.sample_format = "S16_LE"
        self.coding_format = "wave"

```

```

self.channels = 1
self.sample_rate = 16000
self.size = 0
self.priority = 6
self.priority_weight = 0

self.client = self.create_client(PlayAudioFile, self.service_name)
self.get_logger().info("✅ PlayAudioFile client node created.")

def wait_for_service(self) -> bool:
    while not self.client.wait_for_service(timeout_sec=2.0):
        if not rclpy.ok():
            return False
        self.get_logger().info(f"⌚ Waiting for service:
{self.service_name}")
        self.get_logger().info("🟢 Service available, ready to send
request.")
    return True

def send_request(self) -> bool:
    if not self.wait_for_service():
        return False

    try:
        request = PlayAudioFile.Request()
        request.request = CommonRequest()
        request.request.header.stamp = self.get_clock().now().to_msg()

        request.file.pkg_name = self.pkg_name
        request.file.file_name = self.file_name
        request.file.file_path = self.file_path

        if not request.file.file_name:
            self.get_logger().error("❌ file_name is empty.")
            return False

        request.file.info.channels = self.channels
        request.file.info.sample_rate = self.sample_rate
        request.file.info.size = self.size
        request.file.info.sample_format = self.sample_format
        request.file.info.coding_format = self.coding_format
        request.file.priority = self.priority
        request.file.priority_weight = self.priority_weight

        self.get_logger().info(
            "📁 Sending PlayAudioFile request: "

```

```

        f"file={request.file.file_name}, "
        f"path={request.file.file_path}, "
        f"pkg={request.file.pkg_name}, "
        f"ch={self.channels}, "
        f"sr={self.sample_rate}, "
        f"fmt={self.sample_format}, "
        f"coding={self.coding_format}, "
        f"priority={self.priority}, "
        f"weight={self.priority_weight}"
    )

    future = self.client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
    if not future.done():
        self.get_logger().error("❌ PlayAudioFile call failed or
timed out.")
        return False

    response = future.result()
    if response is None:
        self.get_logger().error("❌ PlayAudioFile call failed or
timed out.")
        return False

    code = response.reponse.header.code
    status = response.reponse.status.value
    ok = code == 0 or status == CommonState.SUCCESS

    if ok:
        self.get_logger().info(
            "✅ PlayAudioFile accepted. "
            f"code={code} status={status} msg=
{response.reponse.message}"
        )
        return True

    self.get_logger().error(
        "❌ PlayAudioFile rejected. "
        f"code={code} status={status} msg=
{response.reponse.message}"
    )
    return False
except Exception as error: # noqa: BLE001
    self.get_logger().error(f"❌ Exception occurred: {error}")
    return False

```

```

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = PlayAudioFileClient()
        ok = node.send_request()
        return 0 if ok else 1
    except Exception as error: # noqa: BLE001
        rclpy.logging.get_logger("main").error(
            f"❌ Program exited with exception: {error}"
        )
        return 1
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

代码示例

音频参数控制

音频参数控制 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp audio_control`

```

/**
 * @file audio_control.cpp
 * @brief 启元机器人音频控制示例程序
 *
 * 演示内容：
 *   1. 播放一段较长的 TTS 语音作为背景播报。
 *   2. 在播报过程中，使用定时器按步骤动态调节音量（SetVolume）和静音状态（SetMute）。
 *   3. 每次设置完成后，调发对应的查询接口（GetVolume / GetMute）进行双向回读校验，确保执行结果。
 */

#include "aimdk_msgs/msg/common_request.hpp"
#include "aimdk_msgs/msg/tts_priority_level.hpp"
#include "aimdk_msgs/srv/get_mute.hpp"

```

```

#include "aimdk_msgs/srv/get_volume.hpp"
#include "aimdk_msgs/srv/play_tts.hpp"
#include "aimdk_msgs/srv/set_mute.hpp"
#include "aimdk_msgs/srv/set_volume.hpp"
#include "rclcpp/executors/multi_threaded_executor.hpp"
#include "rclcpp/rclcpp.hpp"

#include <chrono>
#include <csignal>
#include <cstdint>
#include <functional>
#include <future>
#include <memory>
#include <mutex>
#include <string>

using namespace std::chrono_literals;

namespace {

// 背景 TTS 播报文本内容
constexpr char kTtsText[] =
    "大家好，我是启元机器人Q1。现在为你演示音频控制示例。"
    "接下来，我会先持续播报一段介绍内容，在播报过程中，系统会依次执行设置音量、查询音量、"
    "设置静音和查询静音等操作。"
    "你将听到默认音量、音量调小、音量调大、开启静音以及取消静音这几个阶段的变化。"
    "如果整段流程能够顺利完成，就说明启元的TTS播放、音量控制和静音控制链路都已经正常工作。"
    "感谢你体验启元机器人Q1的音频控制能力。";

constexpr auto kStepInterval = 5s;          // 状态机执行步进间隔：5s
constexpr auto kServiceWaitInterval = 2s;   // 等待服务端上线的周期：2s
constexpr auto kServiceCallTimeout = 5s;    // 服务调用超时时长：5s

constexpr std::size_t kTotalSteps = 5;      // 总控制步骤数

} // namespace

class VolumeControlClient : public rclcpp::Node {
public:
    VolumeControlClient() : Node("volume_control_client") {
        // 实例化一个【可重入(Reentrant)】回调并发组
        // 作用：允许定时器/Service客户端的回调可以在多个执行器线程中同时响应，防止死锁
        callback_group_ =
            this->create_callback_group(rclcpp::CallbackGroupType::Reentrant);
    }

```

```

// 1. 注册客户端绑定到并发回调组中
play_tts_client_ = this->create_client<aimdk_msgs::srv::PlayTts>(
    play_tts_service_, rmw_qos_profile_services_default,
callback_group_);
set_volume_client_ = this->create_client<aimdk_msgs::srv::SetVolume>(
    set_volume_service_, rmw_qos_profile_services_default,
callback_group_);
get_volume_client_ = this->create_client<aimdk_msgs::srv::GetVolume>(
    get_volume_service_, rmw_qos_profile_services_default,
callback_group_);
set_mute_client_ = this->create_client<aimdk_msgs::srv::SetMute>(
    set_mute_service_, rmw_qos_profile_services_default,
callback_group_);
get_mute_client_ = this->create_client<aimdk_msgs::srv::GetMute>(
    get_mute_service_, rmw_qos_profile_services_default,
callback_group_);

// 2. 创建一次性的启动定时器, 缓冲节点初启动, 避免 spin 抢跑
startup_timer_ = this->create_wall_timer(
    200ms, std::bind(&VolumeControlClient::start_demo, this),
    callback_group_);

RCLCPP_INFO(this->get_logger(),
    "Volume control demo node created. Waiting to start.");
}

// 服务端全局初始化探针: 验证所有依赖 Service 是否均已上线
bool initialize() {
    return wait_for_service(play_tts_client_, play_tts_service_) &&
        wait_for_service(set_volume_client_, set_volume_service_) &&
        wait_for_service(get_volume_client_, get_volume_service_) &&
        wait_for_service(set_mute_client_, set_mute_service_) &&
        wait_for_service(get_mute_client_, get_mute_service_);
}

// 软关断/节点状态回收控制
void request_stop(int exit_code) {
    {
        std::lock_guard<std::mutex> lock(state_mutex_);
        if (shutdown_requested_) {
            if (exit_code == 0 && exit_code != 0) {
                exit_code_ = exit_code;
            }
        } else {
            shutdown_requested_ = true;
            if (exit_code == 0 || exit_code != 0) {

```

```

        exit_code_ = exit_code;
    }
}

if (startup_timer_) {
    startup_timer_->cancel();
}
if (step_timer_) {
    step_timer_->cancel();
}
}

int exit_code() const {
    std::lock_guard<std::mutex> lock(state_mutex_);
    return exit_code_;
}

void handle_signal(int signal) {
    RCLCPP_INFO(this->get_logger(), "Received signal %d, shutting down...",
                signal);
    request_stop(signal);
}

private:
// 校验底层服务端心跳的通用轮询方法
bool wait_for_service(const rclcpp::ClientBase::SharedPtr &client,
                     const std::string &service_name) {
    while (!client->wait_for_service(kServiceWaitInterval)) {
        if (!rclcpp::ok()) {
            return false;
        }
        RCLCPP_INFO(this->get_logger(), "Waiting for service: %s",
                    service_name.c_str());
    }
    RCLCPP_INFO(this->get_logger(), "Service available: %s",
                service_name.c_str());
    return true;
}

// C++ 泛型高度封装的 Service 调用模版
template <typename ServiceT>
std::shared_ptr<typename ServiceT::Response> call_service(
    const typename rclcpp::Client<ServiceT>::SharedPtr &client,
    const std::shared_ptr<typename ServiceT::Request> &request,
    const char *service_name) {

```

```

    auto future = client->async_send_request(request);
    if (future.wait_for(kServiceCallTimeout) != std::future_status::ready)
    {
        RCLCPP_ERROR(this->get_logger(), "%s timed out after %ld ms.",
                     service_name, kServiceCallTimeout.count());
        return nullptr;
    }
    return future.get();
}

// 演示流程触发逻辑入口
void start_demo() {
    {
        std::lock_guard<std::mutex> lock(state_mutex_);
        if (shutdown_requested_ || started_) {
            return;
        }
        started_ = true;
    }

    startup_timer_>cancel(); // 撤销开机缓存定时器

    RCLCPP_INFO(this->get_logger(),
                "Volume control demo starts automatically.");
    RCLCPP_INFO(this->get_logger(),
                "Volume note: on this device a larger value means a smaller
"
                "actual volume.");

    // 1. 发起 TTS 播报请求先
    if (!play_tts()) {
        fail_and_shutdown("PlayTts failed.");
        return;
    }

    // 2. 挂载主力周期性步进状态机（等5秒走一步控制）
    step_timer_ = this->create_wall_timer(
        kStepInterval, std::bind(&VolumeControlClient::run_step, this),
        callback_group_);

    RCLCPP_INFO(this->get_logger(),
                "TTS accepted. The first control step will run in %ld
seconds.",
                kStepInterval.count());
}

```

```

// 下发时长背景语料
bool play_tts() {
    auto request = std::make_shared<aimdk_msgs::srv::PlayTts::Request>();
    request->header = aimdk_msgs::msg::CommonRequest();
    request->header.header.stamp = this->now();
    request->tts_req.text = kTtsText;
    request->tts_req.domain = tts_domain_;
    request->tts_req.trace_id = tts_trace_id_;
    request->tts_req.is_interrupted = true; // 允许抢占和被打断打断
    request->tts_req.priority_weight = 0;
    request->tts_req.priority_level.value =
        aimdk_msgs::msg::TtsPriorityLevel::INTERACTION_L6;

    RCLCPP_INFO(this->get_logger(),
        "Sending PlayTts request. domain=%s trace_id=%s",
        request->tts_req.domain.c_str(),
        request->tts_req.trace_id.c_str());
    RCLCPP_INFO(this->get_logger(), "TTS text: %s",
        request->tts_req.text.c_str());

    auto response =
        call_service<aimdk_msgs::srv::PlayTts>(play_tts_client_, request,
            "PlayTts");

    if (!response) {
        return false;
    }

    RCLCPP_INFO(this->get_logger(),
        "PlayTts response: code=%ld status=%d is_success=%d "
        "error_message=%s",
        response->header.header.code, response->
>header.status.value,
        static_cast<int>(response->tts_resp.is_success),
        response->tts_resp.error_message.c_str());

    if (!response->tts_resp.is_success) {
        return false;
    }

    return true;
}

// 状态机周期循环处理核心
void run_step() {
    std::size_t current_step = 0;
    {

```

```

std::lock_guard<std::mutex> lock(state_mutex_);
if (shutdown_requested_) {
    return;
}
if (step_in_progress_) {
    RCLCPP_WARN(this->get_logger(),
        "Previous control step is still running, skip this
tick.");
    return;
}
if (step_index_ >= kTotalSteps) {
    return;
}
step_in_progress_ = true;
current_step = step_index_;
}

bool ok = false;
// 根据当前步进数组 index 进行状态选择执行
switch (current_step) {
case 0:
    ok = execute_volume_step(30, "设为默认音量");
    break;
case 1:
    ok = execute_volume_step(40, "音量调小");
    break;
case 2:
    ok = execute_volume_step(20, "音量调大");
    break;
case 3:
    ok = execute_mute_step(true, "设置静音");
    break;
case 4:
    ok = execute_mute_step(false, "取消静音");
    break;
default:
    ok = false;
    break;
}

bool is_done = false;
{
    std::lock_guard<std::mutex> lock(state_mutex_);
    step_in_progress_ = false;
    if (shutdown_requested_) {
        return;
    }
}

```

```

    }
    if (ok) {
        ++step_index_;
        // 结束哨位校验
        is_done = step_index_ >= kTotalSteps;
    }
}

if (!ok) {
    fail_and_shutdown("Volume control step failed.");
    return;
}

if (is_done) {
    RCLCPP_INFO(this->get_logger(),
                "Volume control demo finished. Final state: volume=20, "
                "mute=false.");
    request_stop(0);
    rclcpp::shutdown();
}
}

// 执行音量调节双向校验过程 (SetVolume -> GetVolume -> Contrast)
bool execute_volume_step(std::uint32_t target_volume,
                        const char *action_message) {
    RCLCPP_INFO(this->get_logger(),
                "Running volume step. target=%u action=%s",
                target_volume, action_message);

    // 1. 下发 SetVolume
    auto set_request =
std::make_shared<aimdk_msgs::srv::SetVolume::Request>();
    set_request->request = aimdk_msgs::msg::CommonRequest();
    set_request->request.header.stamp = this->now();
    set_request->audio_volume = target_volume;

    auto set_response = call_service<aimdk_msgs::srv::SetVolume>(
        set_volume_client_, set_request, "SetVolume");
    if (!set_response) {
        return false;
    }

    RCLCPP_INFO(this->get_logger(),
                "SetVolume response: code=%ld status=%d audio_volume=%u",
                set_response->reponse.header.code,
                set_response->reponse.status.value,

```

```

        set_response->audio_volume);

// 2. 下发 GetVolume 读回当前声卡实际输出状态值作对比校验
auto get_request =
std::make_shared<aimdk_msgs::srv::GetVolume::Request>();
get_request->request = aimdk_msgs::msg::CommonRequest();
get_request->request.header.stamp = this->now();

auto get_response = call_service<aimdk_msgs::srv::GetVolume>(
    get_volume_client_, get_request, "GetVolume");
if (!get_response) {
    return false;
}

RCLCPP_INFO(this->get_logger(),
    "GetVolume response: code=%ld status=%d audio_volume=%u",
    get_response->reponse.header.code,
    get_response->reponse.status.value,
    get_response->audio_volume);

if (get_response->audio_volume != target_volume) {
    RCLCPP_ERROR(this->get_logger(),
        "Volume verification failed. target=%u actual=%u",
        target_volume, get_response->audio_volume);
    return false;
}

RCLCPP_INFO(this->get_logger(), "%s, 确认当前音量=%u", action_message,
    get_response->audio_volume);
return true;
}

// 执行静音调节双向校验过程 (SetMute -> GetMute -> Contrast)
bool execute_mute_step(bool target_mute, const char *action_message) {
    RCLCPP_INFO(this->get_logger(), "Running mute step. target=%d
action=%s",
        static_cast<int>(target_mute), action_message);

// 1. 下发 SetMute
auto set_request = std::make_shared<aimdk_msgs::srv::SetMute::Request>
());
set_request->request = aimdk_msgs::msg::CommonRequest();
set_request->request.header.stamp = this->now();
set_request->is_mute = target_mute;

auto set_response = call_service<aimdk_msgs::srv::SetMute>(

```

```

        set_mute_client_, set_request, "SetMute");
    if (!set_response) {
        return false;
    }

    RCLCPP_INFO(
        this->get_logger(),
        "SetMute response: code=%ld status=%d is_mute=%d "
        "(final result is verified by GetMute)",
        set_response->reponse.header.code,
        set_response->reponse.status.value,
        static_cast<int>(set_response->is_mute));

    // 2. 下发 GetMute 做校验比对
    auto get_request = std::make_shared<aimdk_msgs::srv::GetMute::Request>
();
    get_request->request = aimdk_msgs::msg::CommonRequest();
    get_request->request.header.stamp = this->now();

    auto get_response = call_service<aimdk_msgs::srv::GetMute>(
        get_mute_client_, get_request, "GetMute");
    if (!get_response) {
        return false;
    }

    RCLCPP_INFO(this->get_logger(),
        "GetMute response: code=%ld status=%d is_mute=%d",
        get_response->reponse.header.code,
        get_response->reponse.status.value,
        static_cast<int>(get_response->is_mute));

    if (get_response->is_mute != target_mute) {
        RCLCPP_ERROR(this->get_logger(),
            "Mute verification failed. target=%d actual=%d",
            static_cast<int>(target_mute),
            static_cast<int>(get_response->is_mute));
        return false;
    }

    RCLCPP_INFO(this->get_logger(), "%s, 确认当前静音状态=%d", action_message,
        static_cast<int>(get_response->is_mute));
    return true;
}

void fail_and_shutdown(const char *reason) {
    RCLCPP_ERROR(this->get_logger(), "%s", reason);
}

```

```

    request_stop(1);
    rclcpp::shutdown();
}

// 依赖底层接口的服务路由
rclcpp::CallbackGroup::SharedPtr callback_group_;
const std::string play_tts_service_ = "/aimdk_5Fmsgs/srv/PlayTts";
const std::string set_volume_service_ = "/aimdk_5Fmsgs/srv/SetVolume";
const std::string get_volume_service_ = "/aimdk_5Fmsgs/srv/GetVolume";
const std::string set_mute_service_ = "/aimdk_5Fmsgs/srv/SetMute";
const std::string get_mute_service_ = "/aimdk_5Fmsgs/srv/GetMute";
const std::string tts_domain_ = "volume_control_demo";
const std::string tts_trace_id_ = "volume_control_demo_trace";
rclcpp::Client<aimdk_msgs::srv::PlayTts>::SharedPtr play_tts_client_;
rclcpp::Client<aimdk_msgs::srv::SetVolume>::SharedPtr set_volume_client_;
rclcpp::Client<aimdk_msgs::srv::GetVolume>::SharedPtr get_volume_client_;
rclcpp::Client<aimdk_msgs::srv::SetMute>::SharedPtr set_mute_client_;
rclcpp::Client<aimdk_msgs::srv::GetMute>::SharedPtr get_mute_client_;
rclcpp::TimerBase::SharedPtr startup_timer_;
rclcpp::TimerBase::SharedPtr step_timer_;

mutable std::mutex state_mutex_;
std::size_t step_index_ = 0;
bool started_ = false;
bool step_in_progress_ = false;
bool shutdown_requested_ = false;
int exit_code_ = 0;
};

std::shared_ptr<VolumeControlClient> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        g_node->handle_signal(signal);
    }
    rclcpp::shutdown();
}

int main(int argc, char *argv[]) {
    try {
        rclcpp::init(argc, argv);
        std::signal(SIGINT, signal_handler);
        std::signal(SIGTERM, signal_handler);

        g_node = std::make_shared<VolumeControlClient>();
        if (!g_node->initialize()) {

```

```

    RCLCPP_ERROR(rclcpp::get_logger("volume_control_main"),
                  "Failed to initialize volume control demo.");
    const int exit_code = 1;
    g_node.reset();
    rclcpp::shutdown();
    return exit_code;
}

// 使用 2个工作线程 的多线程执行器 (MultiThreadedExecutor)
// 作用: 配合 ReentrantCallbackGroup 使得主循环定时器回调
// 与底层 Service 的反馈回调不互斥, 可在后台线程流畅并进
rclcpp::executors::MultiThreadedExecutor executor(
    rclcpp::ExecutorOptions(), 2);
executor.add_node(g_node);
executor.spin();

const int exit_code = g_node ? g_node->exit_code() : 0;
g_node.reset();
rclcpp::shutdown();
return exit_code;
} catch (const std::exception &e) {
    RCLCPP_ERROR(rclcpp::get_logger("volume_control_main"),
                  "Program exited with exception: %s", e.what());
    return 1;
}
}

```

音频参数控制 Python示例

执行前置: 需先满足 [Python代码运行前置条件](#)

运行命令: `python3 primebot_sdk/examples/python/audio_control.py`

```

#!/usr/bin/env python3

import time

import rclpy
import rclpy.logging
from rclpy.node import Node

from aimdk_msgs.msg import CommonRequest, TtsPriorityLevel
from aimdk_msgs.srv import GetMute, GetVolume, PlayTts, SetMute, SetVolume

TTS_TEXT = (
    "大家好, 我是启元机器人Q1。现在为你演示音频控制示例。"
    "接下来, 我会先持续播报一段介绍内容, 在播报过程中, 系统会依次执行设置音量、查询音量、

```

设置静音和查询静音等操作。"

"你将听到默认音量、音量调小、音量调大、开启静音以及取消静音这几个阶段的变化。"

"如果整段流程能够顺利完成，就说明启元的TTS播放、音量控制和静音控制链路都已经正常工作。"

"感谢你体验启元机器人Q1的音频控制能力。"

)

STEP_INTERVAL_SEC = 5.0

SERVICE_WAIT_SEC = 2.0

SERVICE_CALL_TIMEOUT_SEC = 5.0

```
class AudioControlClient(Node):
```

```
    def __init__(self):
```

```
        super().__init__("audio_control_client")
```

```
        self.play_tts_service = "/aimdk_5Fmsgs/srv/PlayTts"
```

```
        self.set_volume_service = "/aimdk_5Fmsgs/srv/SetVolume"
```

```
        self.get_volume_service = "/aimdk_5Fmsgs/srv/GetVolume"
```

```
        self.set_mute_service = "/aimdk_5Fmsgs/srv/SetMute"
```

```
        self.get_mute_service = "/aimdk_5Fmsgs/srv/GetMute"
```

```
        self.tts_domain = "volume_control_demo"
```

```
        self.tts_trace_id = "volume_control_demo_trace"
```

```
        self.play_tts_client = self.create_client(PlayTts,  
self.play_tts_service)
```

```
        self.set_volume_client = self.create_client(  
            SetVolume, self.set_volume_service  
        )
```

```
        self.get_volume_client = self.create_client(  
            GetVolume, self.get_volume_service  
        )
```

```
        self.set_mute_client = self.create_client(SetMute,  
self.set_mute_service)
```

```
        self.get_mute_client = self.create_client(GetMute,  
self.get_mute_service)
```

```
        self.get_logger().info("Audio control demo node created.")
```

```
    def wait_for_services(self) -> bool:
```

```
        return (
```

```
            self._wait_for_service(self.play_tts_client,  
self.play_tts_service)
```

```
            and self._wait_for_service(  
                self.set_volume_client, self.set_volume_service  
            )
```

```
            and self._wait_for_service(  
                self.get_volume_client, self.get_volume_service  
            )
```

```

        self.get_volume_client, self.get_volume_service
    )
    and self._wait_for_service(self.set_mute_client,
self.set_mute_service)
    and self._wait_for_service(self.get_mute_client,
self.get_mute_service)
    )

def _wait_for_service(self, client, service_name: str) -> bool:
    while not client.wait_for_service(timeout_sec=SERVICE_WAIT_SEC):
        if not rclpy.ok():
            return False
        self.get_logger().info(f"Waiting for service: {service_name}")

    self.get_logger().info(f"Service available: {service_name}")
    return True

def _call_service(self, client, request, service_name: str):
    future = client.call_async(request)
    rclpy.spin_until_future_complete(
        self, future, timeout_sec=SERVICE_CALL_TIMEOUT_SEC
    )
    if not future.done():
        self.get_logger().error(
            f"{service_name} timed out after "
            f"{int(SERVICE_CALL_TIMEOUT_SEC * 1000)} ms."
        )
        return None

    try:
        response = future.result()
    except Exception as error: # noqa: BLE001
        self.get_logger().error(f"{service_name} failed: {error}")
        return None

    if response is None:
        self.get_logger().error(f"{service_name} returned no
response.")
        return None

    return response

def _build_common_request(self) -> CommonRequest:
    request = CommonRequest()
    request.header.stamp = self.get_clock().now().to_msg()
    return request

```

```

def play_tts(self) -> bool:
    request = PlayTts.Request()
    request.header = self._build_common_request()
    request.tts_req.text = TTS_TEXT
    request.tts_req.domain = self.tts_domain
    request.tts_req.trace_id = self.tts_trace_id
    request.tts_req.is_interrupted = True
    request.tts_req.priority_weight = 0
    request.tts_req.priority_level.value =
TtsPriorityLevel.INTERACTION_L6

    self.get_logger().info(
        f"Sending PlayTts request. domain={request.tts_req.domain} "
        f"trace_id={request.tts_req.trace_id}"
    )
    self.get_logger().info(f"TTS text: {request.tts_req.text}")

    response = self._call_service(
        self.play_tts_client, request, "PlayTts"
    )
    if response is None:
        return False

    self.get_logger().info(
        "PlayTts response: "
        f"code={response.header.header.code} "
        f"status={response.header.status.value} "
        f"is_success={int(response.tts_resp.is_success)} "
        f"error_message={response.tts_resp.error_message}"
    )

    return response.tts_resp.is_success

def execute_volume_step(self, target_volume: int, action_message: str)
-> bool:
    self.get_logger().info(
        f"Running volume step. target={target_volume} action=
{action_message}"
    )

    set_request = SetVolume.Request()
    set_request.request = self._build_common_request()
    set_request.audio_volume = target_volume

    set_response = self._call_service(

```

```

        self.set_volume_client, set_request, "SetVolume"
    )
    if set_response is None:
        return False

    self.get_logger().info(
        "SetVolume response: "
        f"code={set_response.reponse.header.code} "
        f"status={set_response.reponse.status.value} "
        f"audio_volume={set_response.audio_volume}"
    )

    get_request = GetVolume.Request()
    get_request.request = self._build_common_request()

    get_response = self._call_service(
        self.get_volume_client, get_request, "GetVolume"
    )
    if get_response is None:
        return False

    self.get_logger().info(
        "GetVolume response: "
        f"code={get_response.reponse.header.code} "
        f"status={get_response.reponse.status.value} "
        f"audio_volume={get_response.audio_volume}"
    )

    if get_response.audio_volume != target_volume:
        self.get_logger().error(
            "Volume verification failed. "
            f"target={target_volume} actual=
{get_response.audio_volume}"
        )
        return False

    self.get_logger().info(
        f"{action_message}, 确认当前音量={get_response.audio_volume}"
    )
    return True

def execute_mute_step(self, target_mute: bool, action_message: str) ->
bool:
    self.get_logger().info(
        f"Running mute step. target={int(target_mute)} action=
{action_message}"
    )

```

```

    )

    set_request = SetMute.Request()
    set_request.request = self._build_common_request()
    set_request.is_mute = target_mute

    set_response = self._call_service(
        self.set_mute_client, set_request, "SetMute"
    )
    if set_response is None:
        return False

    self.get_logger().info(
        "SetMute response: "
        f"code={set_response.reponse.header.code} "
        f"status={set_response.reponse.status.value} "
        f"is_mute={int(set_response.is_mute)} "
        "(final result is verified by GetMute)"
    )

    get_request = GetMute.Request()
    get_request.request = self._build_common_request()

    get_response = self._call_service(self.get_mute_client,
    get_request, "GetMute")
    if get_response is None:
        return False

    self.get_logger().info(
        "GetMute response: "
        f"code={get_response.reponse.header.code} "
        f"status={get_response.reponse.status.value} "
        f"is_mute={int(get_response.is_mute)}"
    )

    if get_response.is_mute != target_mute:
        self.get_logger().error(
            "Mute verification failed. "
            f"target={int(target_mute)} actual=
{int(get_response.is_mute)}"
        )
        return False

    self.get_logger().info(
        f"{action_message}, 确认当前静音状态={int(get_response.is_mute)}"
    )

```

```

        return True

    def run_demo(self) -> bool:
        self.get_logger().info("Volume control demo starts automatically.")
        self.get_logger().info(
            "Volume note: on this device a larger value means a smaller
actual volume."
        )

        if not self.play_tts():
            self.get_logger().error("PlayTts failed.")
            return False

        steps = [
            lambda: self.execute_volume_step(30, "设为默认音量"),
            lambda: self.execute_volume_step(40, "音量调小"),
            lambda: self.execute_volume_step(20, "音量调大"),
            lambda: self.execute_mute_step(True, "设置静音"),
            lambda: self.execute_mute_step(False, "取消静音"),
        ]

        self.get_logger().info(
            f"TTS accepted. The first control step will run in
{int(STEP_INTERVAL_SEC)} seconds."
        )

        for step in steps:
            time.sleep(STEP_INTERVAL_SEC)
            if not step():
                self.get_logger().error("Volume control step failed.")
                return False

        self.get_logger().info(
            "Volume control demo finished. Final state: volume=20,
mute=false."
        )
        return True

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = AudioControlClient()
        if not node.wait_for_services():
            node.get_logger().error("Failed to initialize audio control

```

```

demo.")
    return 1
    ok = node.run_demo()
    return 0 if ok else 1
except Exception as error: # noqa: BLE001
    rclpy.logging.get_logger("audio_control_main").error(
        f"Program exited with exception: {error}"
    )
    return 1
finally:
    if node is not None:
        node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

4.4.2.2 屏幕控制

代码示例

表情播放

表情播放 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp play_emotion`

```

/**
 * @brief Example client for /aimdk_5Fmsgs/srv/PlayEmotion
 *
 * The following ROS parameters can be set via startup arguments:
 * --ros-args -p <name>:=<value>
 *
 * Supported parameters:
 * - type: "emotion" or "file"
 * - emotion_ids: integer array, required when type=emotion
 * - file_paths: string array, required when type=file
 *
 * Examples:
 * ros2 run aimdk_examples_cpp play_emotion --ros-args -p
 * type:=emotion -p emotion_ids:="[90]"
 *
 * ros2 run aimdk_examples_cpp play_emotion --ros-args -p
 * type:=file -p file_paths:="[\"/tmp/demo.mp4\"]"

```

```

*/
#include "aimdk_msgs/msg/common_request.hpp"
#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/srv/play_emotion.hpp"
#include "rclcpp/rclcpp.hpp"

#include <chrono>
#include <stdint>
#include <exception>
#include <memory>
#include <signal.h>
#include <string>
#include <vector>

std::shared_ptr<rclcpp::Node> g_node = nullptr;

void signal_handler(int signal) {
    if (g_node) {
        RCLCPP_INFO(g_node->get_logger(), "Received signal %d, shutting
down...",
                    signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

class PlayEmotionClient : public rclcpp::Node {
public:
    PlayEmotionClient() : Node("play_emotion_client") {
        type_ = this->declare_parameter<std::string>("type", "emotion");
        emotion_ids_ =
            this->declare_parameter<std::vector<int64_t>>>(
                "emotion_ids", std::vector<int64_t>{});
        file_paths_ =
            this->declare_parameter<std::vector<std::string>>>(
                "file_paths", std::vector<std::string>{});

        client_ = this->create_client<aimdk_msgs::srv::PlayEmotion>(
            "/aimdk_5Fmsgs/srv/PlayEmotion");
        RCLCPP_INFO(this->get_logger(), "✅ PlayEmotion client node created.");

        while (!client_->wait_for_service(std::chrono::seconds(2))) {
            if (!rclcpp::ok()) {
                return;
            }
        }
    }

```

```

    RCLCPP_INFO(this->get_logger(), "⌚ Waiting for service...");
}
RCLCPP_INFO(this->get_logger(),
    "🟢 Service available, ready to send request.");
}

bool send_request() {
    try {
        if (!validate_parameters()) {
            return false;
        }

        auto request =
std::make_shared<aimdk_msgs::srv::PlayEmotion::Request>();
        request->header = aimdk_msgs::msg::CommonRequest();
        request->type = type_;
        request->priority = priority_;
        request->loop_count = loop_count_;

        if (type_ == "emotion") {
            request->emotion_ids.reserve(emotion_ids_.size());
            for (const auto emotion_id : emotion_ids_) {
                request->emotion_ids.push_back(static_cast<int32_t>(emotion_id));
            }
        } else {
            request->file_paths = file_paths_;
        }

        request->header.header.stamp = this->now();

        RCLCPP_INFO(this->get_logger(),
            "📬 Sending PlayEmotion request: type=%s emotion_ids=%zu
"
            "file_paths=%zu priority=%d loop_count=%d",
            request->type.c_str(), request->emotion_ids.size(),
            request->file_paths.size(), request->priority,
            request->loop_count);
        if (!request->emotion_ids.empty()) {
            for (const auto emotion_id : request->emotion_ids) {
                RCLCPP_INFO(this->get_logger(), "emotion_id=%d", emotion_id);
            }
        }
        if (!request->file_paths.empty()) {
            for (const auto &file_path : request->file_paths) {
                RCLCPP_INFO(this->get_logger(), "file_path=%s",
file_path.c_str());

```

```

    }
}

auto future = client_->async_send_request(request);
auto retcode = rclcpp::spin_until_future_complete(
    shared_from_this(), future, std::chrono::seconds(2));
if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
    RCLCPP_ERROR(this->get_logger(),
        "❌ Service call failed or timed out.");
    return false;
}

const auto response = future.get();
const auto code = response->header.header.code;
const auto status = response->header.status.value;

RCLCPP_INFO(this->get_logger(),
    "Response: code=%ld status=%d message=%s", code, status,
    response->header.message.c_str());

if (code == 0 || status == aimdk_msgs::msg::CommonState::SUCCESS) {
    RCLCPP_INFO(this->get_logger(), "✅ PlayEmotion request
accepted.");
    return true;
}

RCLCPP_ERROR(this->get_logger(), "❌ PlayEmotion request failed.");
return false;
} catch (const std::exception &e) {
    RCLCPP_ERROR(this->get_logger(), "❌ Exception occurred: %s",
e.what());
    return false;
}
}

private:
bool validate_parameters() {
    if (type_ != "emotion" && type_ != "file") {
        RCLCPP_ERROR(this->get_logger(),
            "❌ Invalid parameter 'type': %s. Use 'emotion' or
'file'.",
            type_.c_str());
        return false;
    }

    if (type_ == "emotion" && emotion_ids_.empty()) {

```

```

        RCLCPP_ERROR(this->get_logger(),
                     "❌ Parameter 'emotion_ids' must be set when
type=emotion.");
        return false;
    }

    if (type_ == "file" && file_paths_.empty()) {
        RCLCPP_ERROR(this->get_logger(),
                     "❌ Parameter 'file_paths' must be set when
type=file.");
        return false;
    }

    return true;
}

std::string type_;
std::vector<int64_t> emotion_ids_;
std::vector<std::string> file_paths_;
int32_t priority_ = 0;
int32_t loop_count_ = 1;
rclcpp::Client<aimdk_msgs::srv::PlayEmotion>::SharedPtr client_;
};

int main(int argc, char *argv[]) {
    try {
        rclcpp::init(argc, argv);
        signal(SIGINT, signal_handler);
        signal(SIGTERM, signal_handler);

        g_node = std::make_shared<PlayEmotionClient>();
        auto client = std::dynamic_pointer_cast<PlayEmotionClient>(g_node);
        const bool ok = client ? client->send_request() : false;

        g_node.reset();
        rclcpp::shutdown();
        return ok ? 0 : 1;
    } catch (const std::exception &e) {
        RCLCPP_ERROR(rclcpp::get_logger("main"),
                     "❌ Program exited with exception: %s", e.what());
        return 1;
    }
}

```

表情播放 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/play_emotion.py`

```
#!/usr/bin/env python3

import rclpy
import rclpy.logging
from rclpy.node import Node
from rclpy.parameter import Parameter

from aimdk_msgs.msg import CommonRequest, CommonState
from aimdk_msgs.srv import PlayEmotion

class PlayEmotionClient(Node):
    def __init__(self):
        super().__init__("play_emotion_client")
        self.declare_parameter("type", "emotion")
        self.declare_parameter("emotion_ids", Parameter.Type.INTEGER_ARRAY)
        self.declare_parameter("file_paths", Parameter.Type.STRING_ARRAY)

        self.type = self.get_parameter("type").value
        self.emotion_ids = self.get_parameter_or(
            "emotion_ids",
            Parameter("emotion_ids", Parameter.Type.INTEGER_ARRAY, []),
        ).value
        self.file_paths = self.get_parameter_or(
            "file_paths",
            Parameter("file_paths", Parameter.Type.STRING_ARRAY, []),
        ).value
        self.priority = 0
        self.loop_count = 1

        self.client = self.create_client(PlayEmotion,
            "/aimdk_5Fmsgs/srv/PlayEmotion")
        self.get_logger().info("✅ PlayEmotion client node created.")

        while not self.client.wait_for_service(timeout_sec=2.0):
            self.get_logger().info("⌚ Waiting for service...")
        self.get_logger().info("🟢 Service available, ready to send request.")

    def send_request(self) -> bool:
        try:
            if not self.validate_parameters():
```

```

        return False

    request = PlayEmotion.Request()
    request.header = CommonRequest()
    request.header.header.stamp = self.get_clock().now().to_msg()
    request.type = self.type
    request.priority = self.priority
    request.loop_count = self.loop_count

    if self.type == "emotion":
        request.emotion_ids = list(self.emotion_ids)
    else:
        request.file_paths = list(self.file_paths)

    self.get_logger().info(
        "📬 Sending PlayEmotion request: "
        f"type={request.type}, "
        f"emotion_ids={list(request.emotion_ids)}, "
        f"file_paths={list(request.file_paths)}, "
        f"priority={request.priority}, "
        f"loop_count={request.loop_count}"
    )
    for emotion_id in request.emotion_ids:
        self.get_logger().info(f"emotion_id={emotion_id}")
    for file_path in request.file_paths:
        self.get_logger().info(f"file_path={file_path}")

    future = self.client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)
    if not future.done():
        self.get_logger().error("❌ Service call failed or timed
out.")

        return False

    response = future.result()
    if response is None:
        self.get_logger().error("❌ Service call failed or timed
out.")

        return False

    code = response.header.header.code
    status = response.header.status.value
    self.get_logger().info(
        "Response: "
        f"code={code}, "
        f"status={status}, "

```

```

        f"message={response.header.message}"
    )

    if code == 0 or status == CommonState.SUCCESS:
        self.get_logger().info("✅ PlayEmotion request accepted.")
        return True

    self.get_logger().error("❌ PlayEmotion request failed.")
    return False
except Exception as error: # noqa: BLE001
    self.get_logger().error(f"❌ Exception occurred: {error}")
    return False

def validate_parameters(self) -> bool:
    if self.type not in ("emotion", "file"):
        self.get_logger().error(
            f"❌ Invalid parameter 'type': {self.type}. Use 'emotion'
or 'file'."
        )
        return False

    if self.type == "emotion" and not self.emotion_ids:
        self.get_logger().error(
            "❌ Parameter 'emotion_ids' must be set when type=emotion."
        )
        return False

    if self.type == "file" and not self.file_paths:
        self.get_logger().error(
            "❌ Parameter 'file_paths' must be set when type=file."
        )
        return False

    return True

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = PlayEmotionClient()
        ok = node.send_request()
        return 0 if ok else 1
    except Exception as error: # noqa: BLE001
        rclpy.logging.get_logger("main").error(
            f"❌ Program exited with exception: {error}"

```

```

    )
    return 1
finally:
    if node is not None:
        node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

4.4.3 硬件抽象模块

4.4.3.1 触觉传感器

代码示例

读取触觉状态

读取触觉状态 Cpp示例

*执行前置：*需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp get_touch_state`

```

#include "aimdk_msgs/msg/touch_state.hpp"
#include "rclcpp/rclcpp.hpp"

#include <iomanip>
#include <sstream>

class TouchStateEcho : public rclcpp::Node {
public:
    TouchStateEcho() : Node("get_touch_state") {
        auto qos = rclcpp::SensorDataQoS();
        sub_ = this->create_subscription<aimdk_msgs::msg::TouchState>(
            "/aima/hal/touch/state", qos,
            std::bind(&TouchStateEcho::callback, this, std::placeholders::_1));

        RCLCPP_INFO(this->get_logger(), "Subscribing touch topic: %s",
            "/aima/hal/touch/state");
    }

private:
    void callback(const aimdk_msgs::msg::TouchState::SharedPtr msg) {
        std::ostringstream oss;
        oss << std::fixed << std::setprecision(6);
    }
}

```

```

    oss << "TouchState received\n"
    << "   frame_id: " << msg->header.frame_id << "\n"
    << "   sequence: " << msg->header.sequence << "\n"
    << "   stamp:      " << rclcpp::Time(msg->header.stamp).seconds() << "
s\n"
    << "   meas:      " << rclcpp::Time(msg->header.meas_stamp).seconds()
<< " s\n"
    << "   event:      " << static_cast<int>(msg->event_type);

    RCLCPP_INFO_THROTTLE(this->get_logger(), *this->get_clock(), 500, "%s",
                        oss.str().c_str());
}
rclcpp::Subscription<aimdk_msgs::msg::TouchState>::SharedPtr sub_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<TouchStateEcho>());
    rclcpp::shutdown();
    return 0;
}

```

读取触觉状态 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/get_touch_state.py`

```

#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data

from aimdk_msgs.msg import TouchState

class TouchStateEcho(Node):
    def __init__(self):
        super().__init__("get_touch_state")

        self.subscription = self.create_subscription(
            TouchState,
            "/aima/hal/touch/state",
            self.callback,
            qos_profile_sensor_data,
        )

```

```

        self.get_logger().info("Subscribing touch topic:
/aima/hal/touch/state")

    @staticmethod
    def _to_seconds(stamp) -> float:
        return float(stamp.sec) + float(stamp.nanosec) / 1_000_000_000.0

    def callback(self, msg: TouchState):
        lines = [
            "TouchState received",
            f"  frame_id: {msg.header.frame_id}",
            f"  sequence: {msg.header.sequence}",
            f"  stamp:      {self._to_seconds(msg.header.stamp):.6f} s",
            f"  meas:       {self._to_seconds(msg.header.meas_stamp):.6f} s",
            f"  event:      {int(msg.event_type)}",
        ]

        self.get_logger().info("\n".join(lines))

def main(args=None):
    rclpy.init(args=args)
    node = TouchStateEcho()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        return 0
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

4.4.3.3 电源管理

代码示例

读取电源状态

读取电源状态 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp get_pmu_state`

```

#include "aimdk_msgs/msg/pmu_state.hpp"
#include "rclcpp/rclcpp.hpp"

#include <iomanip>
#include <sstream>

class PmuStateEcho : public rclcpp::Node {
public:
    PmuStateEcho() : Node("get_pmu_state") {
        auto qos = rclcpp::SensorDataQoS();
        sub_ = this->create_subscription<aimdk_msgs::msg::PmuState>(
            "/aima/hal/pmu/state", qos,
            std::bind(&PmuStateEcho::callback, this, std::placeholders::_1));

        RCLCPP_INFO(this->get_logger(), "Subscribing pmu topic: %s",
            "/aima/hal/pmu/state");
    }

private:
    void callback(const aimdk_msgs::msg::PmuState::SharedPtr msg) {
        std::ostringstream oss;
        oss << std::fixed << std::setprecision(3);
        oss << "PmuState received\n";

        oss << "  pmu_protocol_version: " << msg->pmu_protocol_version << "\n"
            << "  pmu_hardware_version: " << msg->pmu_hardware_version << "\n"
            << "  pmu_software_version: " << msg->pmu_software_version << "\n"
            << "  pmu_bool_status: " << msg->pmu_bool_status << "\n";

        oss << "  soc_5v_voltage:      " << msg->soc_5v_voltage << " V\n"
            << "  sys_5v_voltage:      " << msg->sys_5v_voltage << " V\n"
            << "  sys_12v_voltage:     " << msg->sys_12v_voltage << " V\n"
            << "  fore_leg_48v_voltage: " << msg->fore_leg_48v_voltage << "
V\n"
            << "  hind_leg_48v_voltage: " << msg->hind_leg_48v_voltage << "
V\n"
            << "  ext_12v_voltage:     " << msg->ext_12v_voltage << " V\n"
            << "  ext_24v_voltage:     " << msg->ext_24v_voltage << " V\n"
            << "  soc_5v_current:      " << msg->soc_5v_current << " A\n"
            << "  sys_5v_current:      " << msg->sys_5v_current << " A\n"
            << "  sys_12v_current:     " << msg->sys_12v_current << " A\n"
            << "  fore_leg_48v_current: " << msg->fore_leg_48v_current << "
A\n"
            << "  hind_leg_48v_current: " << msg->hind_leg_48v_current << "
A\n"
            << "  ext_12v_current:     " << msg->ext_12v_current << " A\n"

```

```

        << "   ext_24v_current:           " << msg->ext_24v_current << " A\n";

    oss << "   bms_manufacturer:           " << msg->bms_manufacturer
<< "\n"
        << "   bms_serial_number:         " << msg->bms_serial_number
<< "\n"
        << "   bms_protocol_version:       " << msg-
>bms_protocol_version
        << "\n"
        << "   bms_hardware_version:         " << msg-
>bms_hardware_version
        << "\n"
        << "   bms_software_version:        " << msg-
>bms_software_version
        << "\n"
        << "   bms_status_bits:             " << msg->bms_status_bits
        << "\n";

    oss << "   battery_balance_line_resistance: "
        << msg->battery_balance_line_resistance << " mOhm\n"
        << "   battery_pack_voltage:         " << msg-
>battery_pack_voltage
        << " V\n"
        << "   battery_current:               " << msg->battery_current
        << " A\n"
        << "   battery_output_power:          " << msg-
>battery_output_power
        << " W\n"
        << "   battery_temperature:           " << msg-
>battery_temperature
        << " °C\n"
        << "   battery_remaining_capacity:      "
        << msg->battery_remaining_capacity << " mAh\n"
        << "   battery_remaining_capacity_pct:  "
        << static_cast<int>(msg->battery_remaining_capacity_percentage)
        << " %\n"
        << "   battery_cycle_count:             " << msg-
>battery_cycle_count
        << "\n"
        << "   battery_cycle_total_capacity:     "
        << msg->battery_cycle_total_capacity << " Ah\n";

    RCLCPP_INFO(this->get_logger(), "%s", oss.str().c_str());
}
rclcpp::Subscription<aimdk_msgs::msg::PmuState>::SharedPtr sub_;
};

```

```
int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<PmuStateEcho>());
    rclcpp::shutdown();
    return 0;
}
```

读取电源状态 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 primebot_sdk/examples/python/get_pmu_state.py`

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data

from aimdk_msgs.msg import PmuState

class PmuStateEcho(Node):
    def __init__(self):
        super().__init__("get_pmu_state")

        self.subscription = self.create_subscription(
            PmuState,
            "/aima/hal/pmu/state",
            self.callback,
            qos_profile_sensor_data,
        )

        self.get_logger().info("Subscribing pmu topic: /aima/hal/pmu/state")

    @staticmethod
    def _fmt3(value) -> str:
        return f"{float(value):.3f}"

    def callback(self, msg: PmuState):
        lines = [
            "PmuState received",
            f"  pmu_protocol_version: {msg.pmu_protocol_version}",
            f"  pmu_hardware_version: {msg.pmu_hardware_version}",
            f"  pmu_software_version: {msg.pmu_software_version}",
        ]
```

```

        f" pmu_bool_status: {msg.pmu_bool_status}",
        f" soc_5v_voltage:      {self._fmt3(msg.soc_5v_voltage)} V",
        f" sys_5v_voltage:      {self._fmt3(msg.sys_5v_voltage)} V",
        f" sys_12v_voltage:     {self._fmt3(msg.sys_12v_voltage)} V",
        f" fore_leg_48v_voltage:
{self._fmt3(msg.fore_leg_48v_voltage)} V",
        f" hind_leg_48v_voltage:
{self._fmt3(msg.hind_leg_48v_voltage)} V",
        f" ext_12v_voltage:      {self._fmt3(msg.ext_12v_voltage)} V",
        f" ext_24v_voltage:      {self._fmt3(msg.ext_24v_voltage)} V",
        f" soc_5v_current:       {self._fmt3(msg.soc_5v_current)} A",
        f" sys_5v_current:       {self._fmt3(msg.sys_5v_current)} A",
        f" sys_12v_current:      {self._fmt3(msg.sys_12v_current)} A",
        f" fore_leg_48v_current:
{self._fmt3(msg.fore_leg_48v_current)} A",
        f" hind_leg_48v_current:
{self._fmt3(msg.hind_leg_48v_current)} A",
        f" ext_12v_current:      {self._fmt3(msg.ext_12v_current)} A",
        f" ext_24v_current:      {self._fmt3(msg.ext_24v_current)} A",
        f" bms_manufacturer:      {msg.bms_manufacturer}",
        f" bms_serial_number:     {msg.bms_serial_number}",
        f" bms_protocol_version:
{msg.bms_protocol_version}",
        f" bms_hardware_version:
{msg.bms_hardware_version}",
        f" bms_software_version:
{msg.bms_software_version}",
        f" bms_status_bits:      {msg.bms_status_bits}",
        (
            " battery_balance_line_resistance: "
            f"{msg.battery_balance_line_resistance} mOhm"
        ),
        f" battery_pack_voltage:
{self._fmt3(msg.battery_pack_voltage)} V",
        f" battery_current:
{self._fmt3(msg.battery_current)} A",
        f" battery_output_power:
{self._fmt3(msg.battery_output_power)} W",
        f" battery_temperature:
{self._fmt3(msg.battery_temperature)} degC",
        f" battery_remaining_capacity:
{msg.battery_remaining_capacity} mAh",
        (
            " battery_remaining_capacity_pct: "
            f"{int(msg.battery_remaining_capacity_percentage)} %"
        ),

```

```

        f" battery_cycle_count:
{msg.battery_cycle_count}",
        f" battery_cycle_total_capacity:
{msg.battery_cycle_total_capacity} Ah",
    ]

    self.get_logger().info("\n".join(lines))

def main(args=None):
    rclpy.init(args=args)
    node = PmuStateEcho()
    try:
        rclpy.spin(node)
        return 0
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    raise SystemExit(main())

```

4.4.3.4 灯语控制

代码示例

灯语控制

灯语控制 Cpp示例

执行前置：需先满足 [Cpp代码运行前置条件](#)

运行命令： `ros2 run aimdk_examples_cpp play_lights`

```

#include "aimdk_msgs/msg/common_request.hpp"
#include "aimdk_msgs/msg/common_state.hpp"
#include "aimdk_msgs/srv/set_rgb_strip.hpp"
#include "rclcpp/rclcpp.hpp"

#include <chrono>
#include <cstdlib>
#include <exception>
#include <iostream>
#include <memory>
#include <signal.h>

std::shared_ptr<rclcpp::Node> g_node = nullptr;

```

```

void signal_handler(int signal) {
    if (g_node) {
        RCLCPP_INFO(g_node->get_logger(), "Received signal %d, shutting
down...",
                    signal);
        g_node.reset();
    }
    rclcpp::shutdown();
    exit(signal);
}

class PlayLightsClient : public rclcpp::Node {
public:
    PlayLightsClient() : Node("play_lights_client") {
        client_ = this->create_client<aimdk_msgs::srv::SetRgbStrip>(
            "/aimdk_5Fmsgs/srv/SetRgbStrip");
        RCLCPP_INFO(this->get_logger(), "✅ SetRgbStrip client node created.");

        while (!client_->wait_for_service(std::chrono::seconds(2))) {
            if (!rclcpp::ok()) {
                return;
            }
            RCLCPP_INFO(this->get_logger(), "⌚ Service unavailable,
waiting...");
        }
        RCLCPP_INFO(this->get_logger(),
                    "🟢 Service available, ready to send request.");
    }

    bool send_request(uint8_t led_strip_mode) {
        try {
            auto request =
std::make_shared<aimdk_msgs::srv::SetRgbStrip::Request>();
            request->request = aimdk_msgs::msg::CommonRequest();
            request->led_strip_mode = led_strip_mode;

            RCLCPP_INFO(this->get_logger(),
                        "✉ Sending SetRgbStrip request: led_strip_mode=%u",
                        static_cast<unsigned int>(request->led_strip_mode));

            const std::chrono::milliseconds timeout(2000);
            request->request.header.stamp = this->now();
            auto future = client_->async_send_request(request);
            auto retcode = rclcpp::spin_until_future_complete(
                shared_from_this(), future, timeout);

```

```

    if (retcode != rclcpp::FutureReturnCode::SUCCESS) {
        RCLCPP_ERROR(this->get_logger(),
            "✗ SetRgbStrip service timeout after %ld ms.",
            timeout.count());
        return false;
    }

    auto response = future.get();
    const auto code = response->response.header.code;
    const auto status = response->response.status.value;
    RCLCPP_INFO(this->get_logger(),
        "Response: code=%ld, status=%d, message=%s, result=%u",
        code, status, response->response.message.c_str(),
        static_cast<unsigned int>(response->result));

    if (code == 0 || status == aimdk_msgs::msg::CommonState::SUCCESS) {
        RCLCPP_INFO(this->get_logger(), "✅ SetRgbStrip request
accepted.");
        return true;
    }

    RCLCPP_ERROR(this->get_logger(), "✗ SetRgbStrip request failed.");
    return false;
} catch (const std::exception &e) {
    RCLCPP_ERROR(this->get_logger(), "Exception occurred: %s", e.what());
    return false;
}
}

private:
    rclcpp::Client<aimdk_msgs::srv::SetRgbStrip>::SharedPtr client_;
};

int main(int argc, char *argv[]) {
    try {
        rclcpp::init(argc, argv);
        signal(SIGINT, signal_handler);
        signal(SIGTERM, signal_handler);

        uint8_t led_strip_mode =
            aimdk_msgs::srv::SetRgbStrip::Request::LED_WHITE_ON;

        int mode_input = 0;
        std::cout << "Enter led_strip_mode (default "
            << static_cast<int>(led_strip_mode) << "): ";
        std::cin >> mode_input;
    }
}

```

```

led_strip_mode = static_cast<uint8_t>(mode_input);

g_node = std::make_shared<PlayLightsClient>();
auto client = std::dynamic_pointer_cast<PlayLightsClient>(g_node);
bool ok = false;
if (client) {
    ok = client->send_request(led_strip_mode);
}

g_node.reset();
rclcpp::shutdown();
return ok ? 0 : 1;
} catch (const std::exception &e) {
    RCLCPP_ERROR(rclcpp::get_logger("main"),
        "Program exited with exception: %s", e.what());
    return 1;
}
}

```

灯语控制 Python示例

执行前置：需先满足 [Python代码运行前置条件](#)

运行命令： `python3 set_rgb_strip_client.py 17`

```

import sys
import rclpy
from rclpy.node import Node
from aimdk_msgs.srv import SetRgbStrip
from aimdk_msgs.msg import RequestHeader

class SetRgbStripClient(Node):
    def __init__(self):
        super().__init__('set_rgb_strip_client')
        # Wait for service
        self.client = self.create_client(SetRgbStrip,
            '/aimdk_5Fmsgs/srv/SetRgbStrip')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Service not available, waiting
again...')
        self.get_logger().info('✅ SetRgbStrip service connected.')

    def send_request(self, led_mode):
        req = SetRgbStrip.Request()
        req.request = RequestHeader() # Standard empty request header
        req.led_strip_mode = led_mode

```

```

        future = self.client.call_async(req)
        return future

def main():
    rclpy.init()
    node = SetRgbStripClient()

    # Check enumerations: 17 for LED_WHITE_ON
    led_mode = 17
    if len(sys.argv) > 1:
        led_mode = int(sys.argv[1])

    node.get_logger().info(f'Sending request for LED Mode: {led_mode}')
    future = node.send_request(led_mode)
    rclpy.spin_until_future_complete(node, future)

    try:
        response = future.result()
        node.get_logger().info('✅ Request executed successfully.')
    except Exception as e:
        node.get_logger().error(f'❌ Service call failed: {e}')

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```